

CSE 6220/CX 4220

Introduction to HPC

Lecture 11: Bitonic Sorting

Helen Xu

hxu615@gatech.edu



Georgia Tech College of Computing
School of Computational
Science and Engineering

Parallel Distributed-Memory Sorting

Which existing sequential algorithm can serve as a good starting point?

- $O(n^2)$: Bubble Sort, Insertion Sort etc.
- $O(n \lg n)$: Heap Sort, Merge Sort, etc.

Divide and conquer fits well with parallel processing, so let's start with

Merge Sort

- Divide the list into two equal size lists, sort them, and merge

$$T(n) = 2T(n/2) + O(n) = O(n \lg n)$$

Merge Sort

Distributed-Memory Parallel Merge Sort

- $T(n, p) = T\left(\frac{n}{2}, \frac{p}{2}\right) + O(n)$

As we discussed before, the merge needs to be **parallelized** to make good use of the processors.

- $T(n, p) = T\left(\frac{n}{4}, \frac{p}{4}\right) + O\left(\frac{n}{2}\right) + O(n)$

⇓

- $T(n, p) = T\left(\frac{n}{p}, 1\right) + O\left(n + \frac{n}{2} + \frac{n}{4} + \dots + \frac{n}{p/2}\right)$

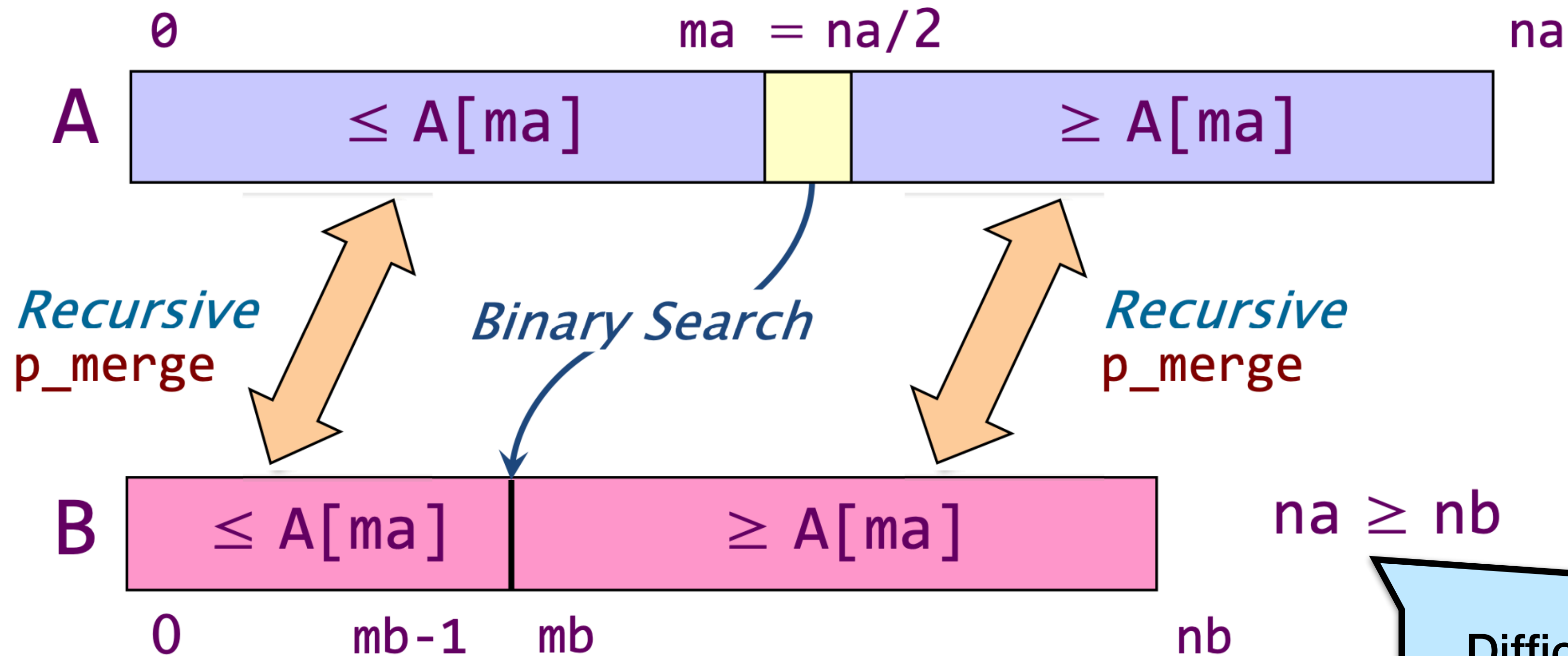
- $T(n, p) = O\left(\frac{n}{p} \log \frac{n}{p} + n\right)$

Time to merge across procs

Time to sort on one processor

Can't use more than $\log n$ processors!

Recall: Parallel Merge



KEY IDEA: If the total number of elements to be merged in the two arrays is $n = na + nb$, the total number of elements in the larger of the two recursive merges is at most $(3/4)n$.

Difficult for distributed memory because the subproblems are uneven - **load balancing** is hard

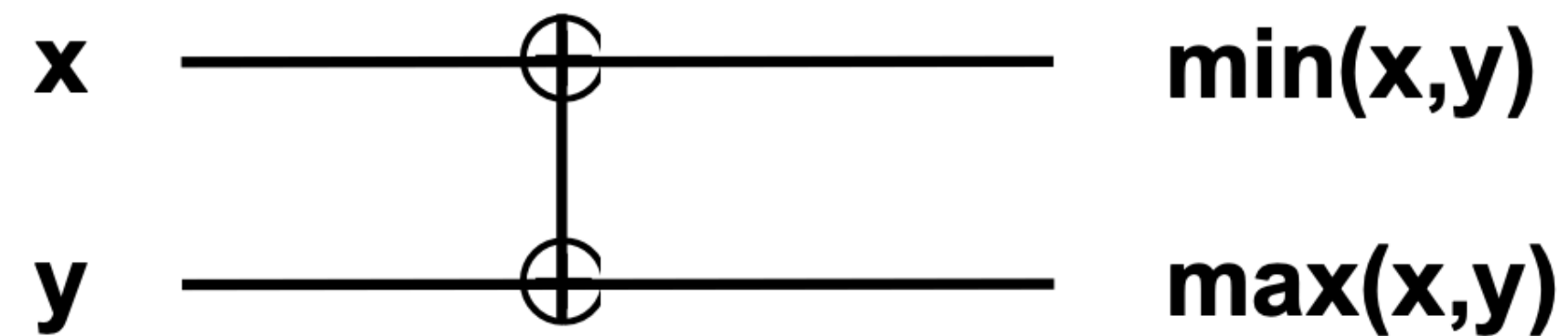
Bitonic Sort

Sorting Network

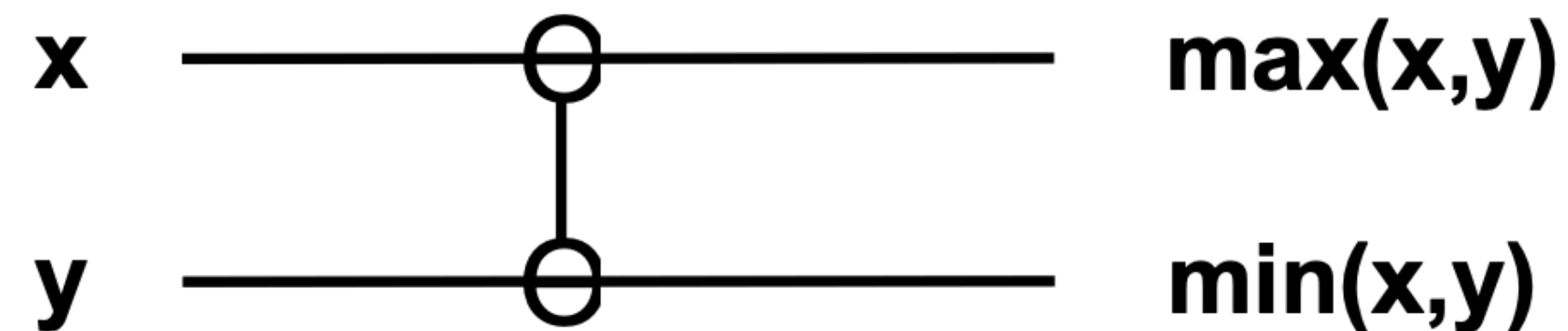
- Network of comparators designed for sorting
- Comparator : two inputs x and y ; two outputs x' and y'

—types

- increasing (denoted $\oplus$): $x' = \min(x,y)$ and $y' = \max(x,y)$



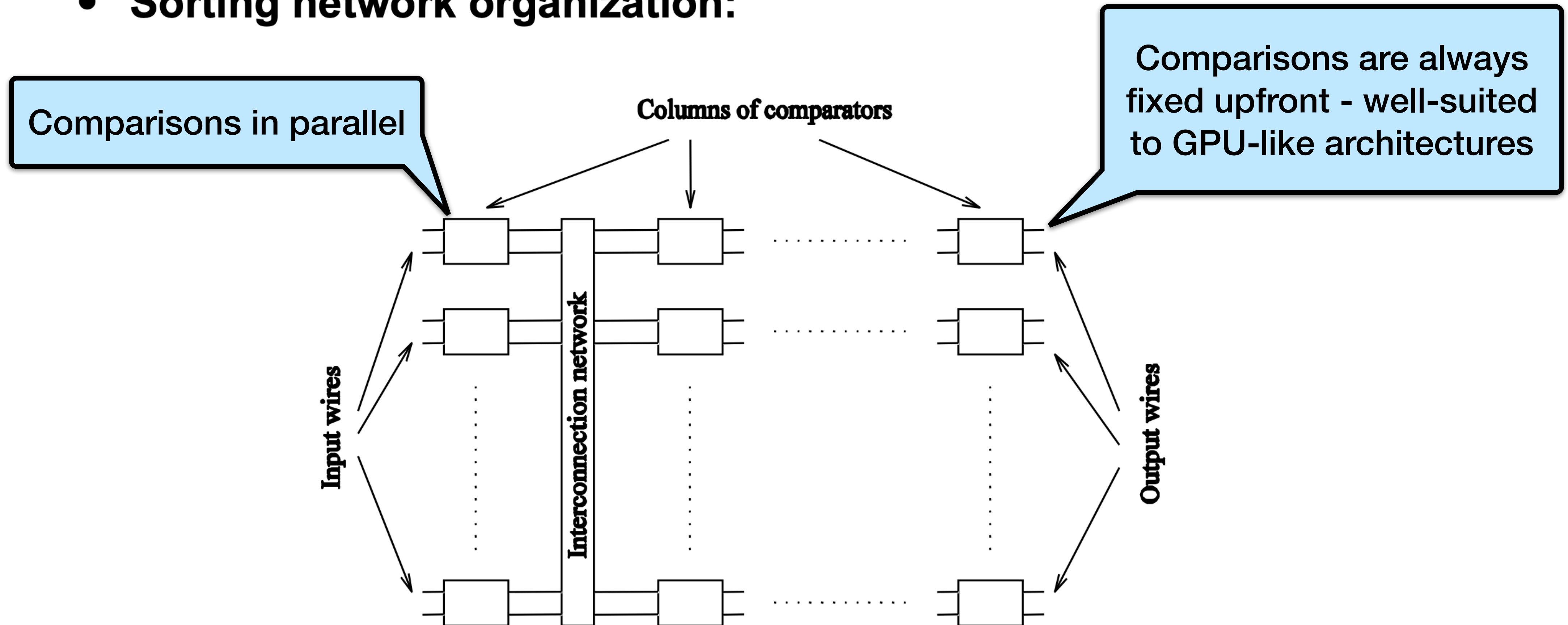
- decreasing (denoted $\ominus$) : $x' = \max(x,y)$ and $y' = \min(x,y)$



- Sorting network speed is proportional to its depth

Sorting Network

- **Network structure: a series of columns**
- **Each column consists of a vector of comparators (in parallel)**
- **Sorting network organization:**



Example: Bitonic Sorting Network

Bitonic sequence

- two parts: increasing and decreasing
 $\langle 1, 2, 4, 7, 6, 0 \rangle$: first increases and then decreases (or vice versa)
- cyclic rotation of a bitonic sequence is also considered bitonic
 $\langle 8, 9, 2, 1, 0, 4 \rangle$: cyclic rotation of $\langle 0, 4, 8, 9, 2, 1 \rangle$

Bitonic sorting network

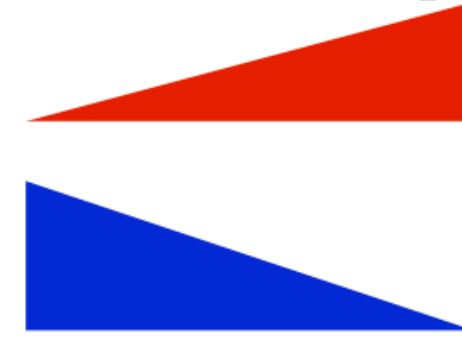
- sorts n elements in $\Theta(\lg^2 n)$ time
- network kernel: rearranges a bitonic sequence **into a sorted one**

Bitonic Split (Batcher, 1968)

- Let $s = \langle a_0, a_1, \dots, a_{n-1} \rangle$ be a bitonic sequence such that

— $a_0 \leq a_1 \leq \dots \leq a_{n/2-1}$, and

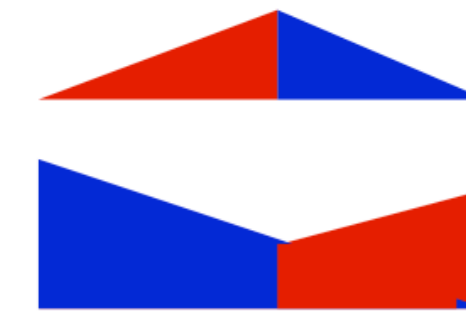
— $a_{n/2} \geq a_{n/2+1} \geq \dots \geq a_{n-1}$



- Consider the following subsequences of s

$$s_1 = \langle \min(a_0, a_{n/2}), \min(a_1, a_{n/2+1}), \dots, \min(a_{n/2-1}, a_{n-1}) \rangle$$

$$s_2 = \langle \max(a_0, a_{n/2}), \max(a_1, a_{n/2+1}), \dots, \max(a_{n/2-1}, a_{n-1}) \rangle$$



- Sequence properties

— s_1 and s_2 are both bitonic

— $\forall x \forall y x \in s_1, y \in s_2, x < y$

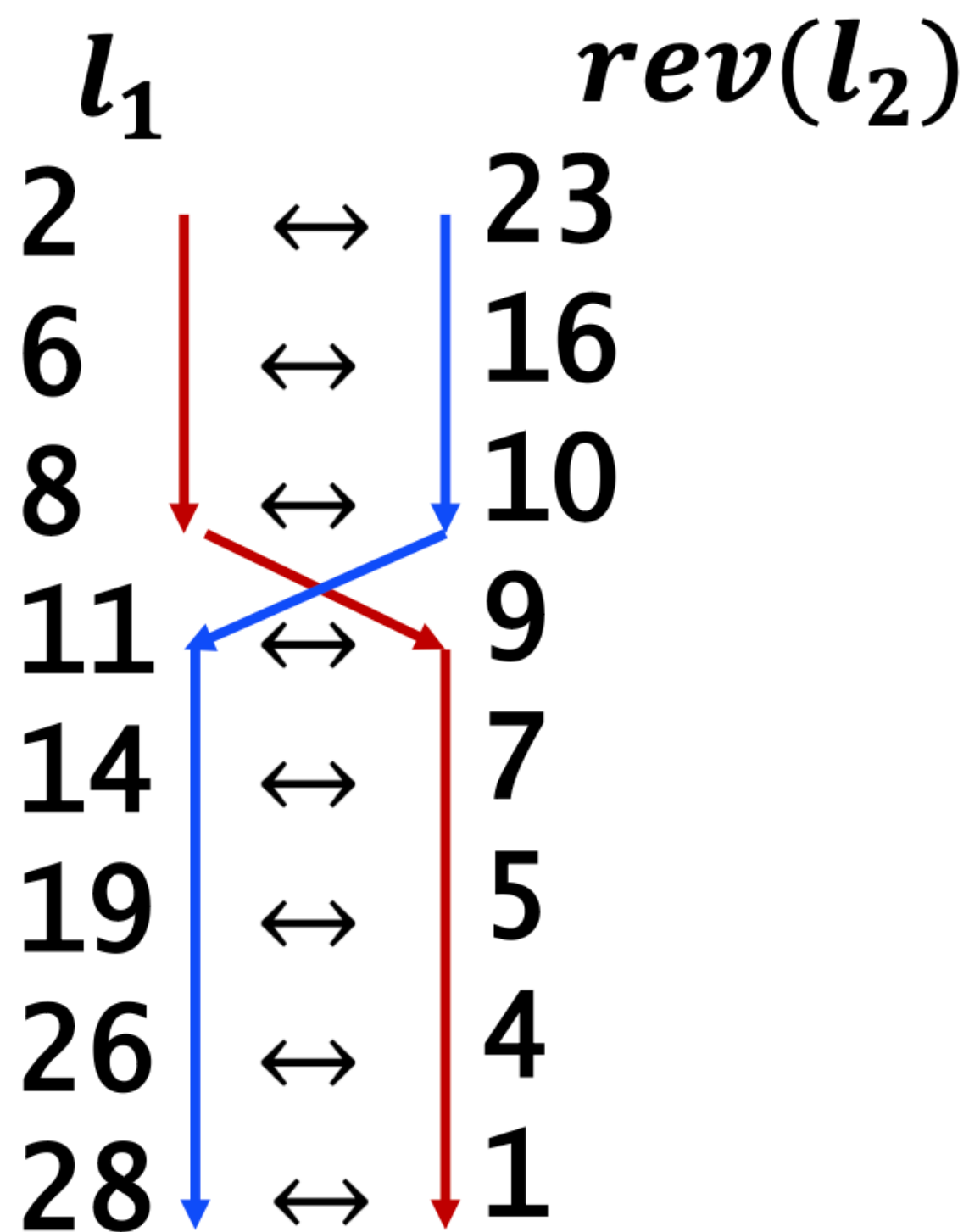
This class

- Apply recursively on s_1 and s_2 to produce a sorted sequence
- Works for any bitonic sequence, even if $|s_1| \neq |s_2|$

Example: Bitonic Split

- l_1 : 2, 6, 8, 11, 14, 19, 26, 28
- l_2 : 1, 4, 5, 7, 9, 10, 16, 23

Non-decreasing
sorted lists



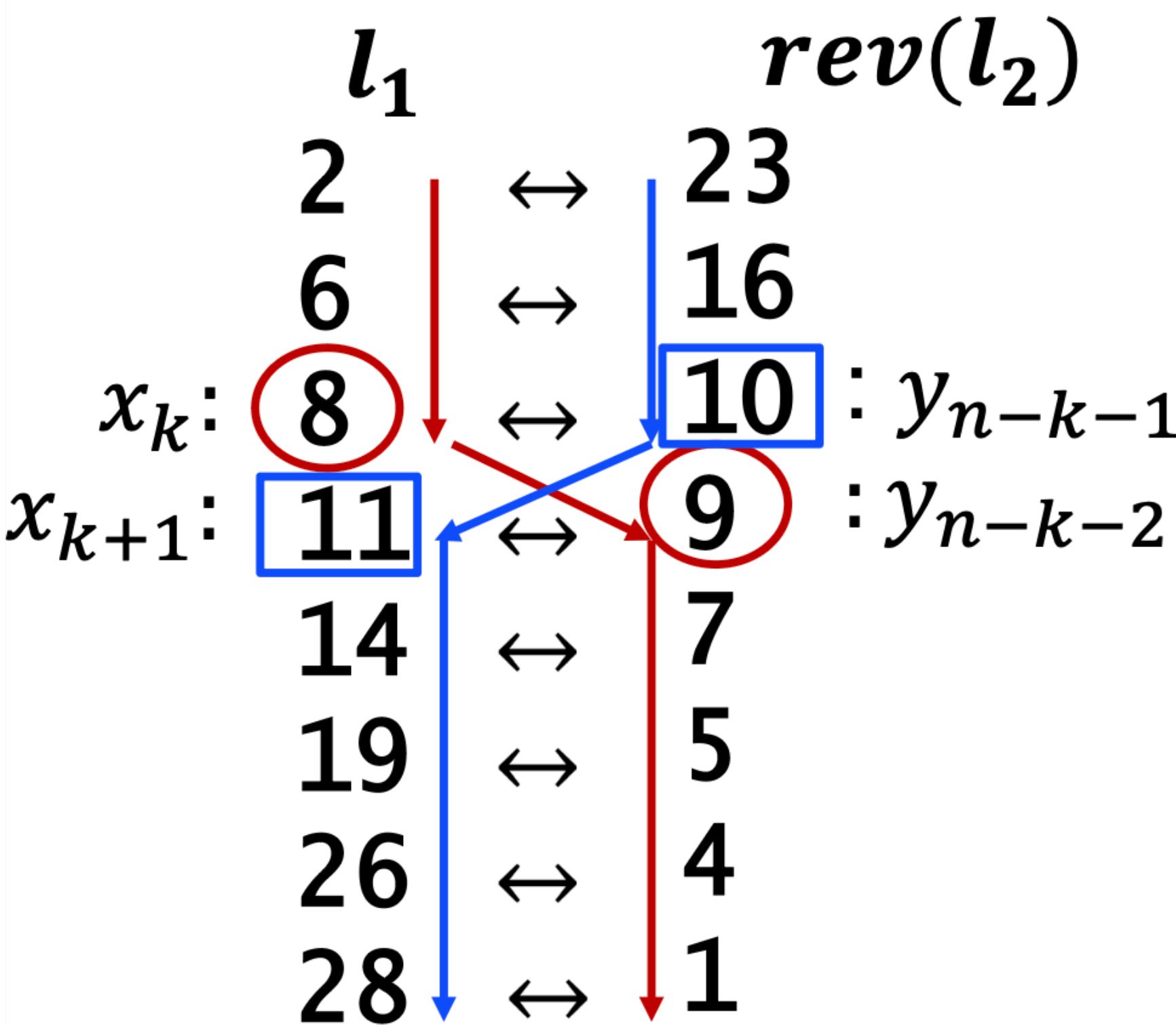
l_{min}	l_{max}
2	23
6	16
8	10
9	11
7	14
5	19
4	26
1	28

How to prove this?

All elements of l_{min} are smaller than l_{max}

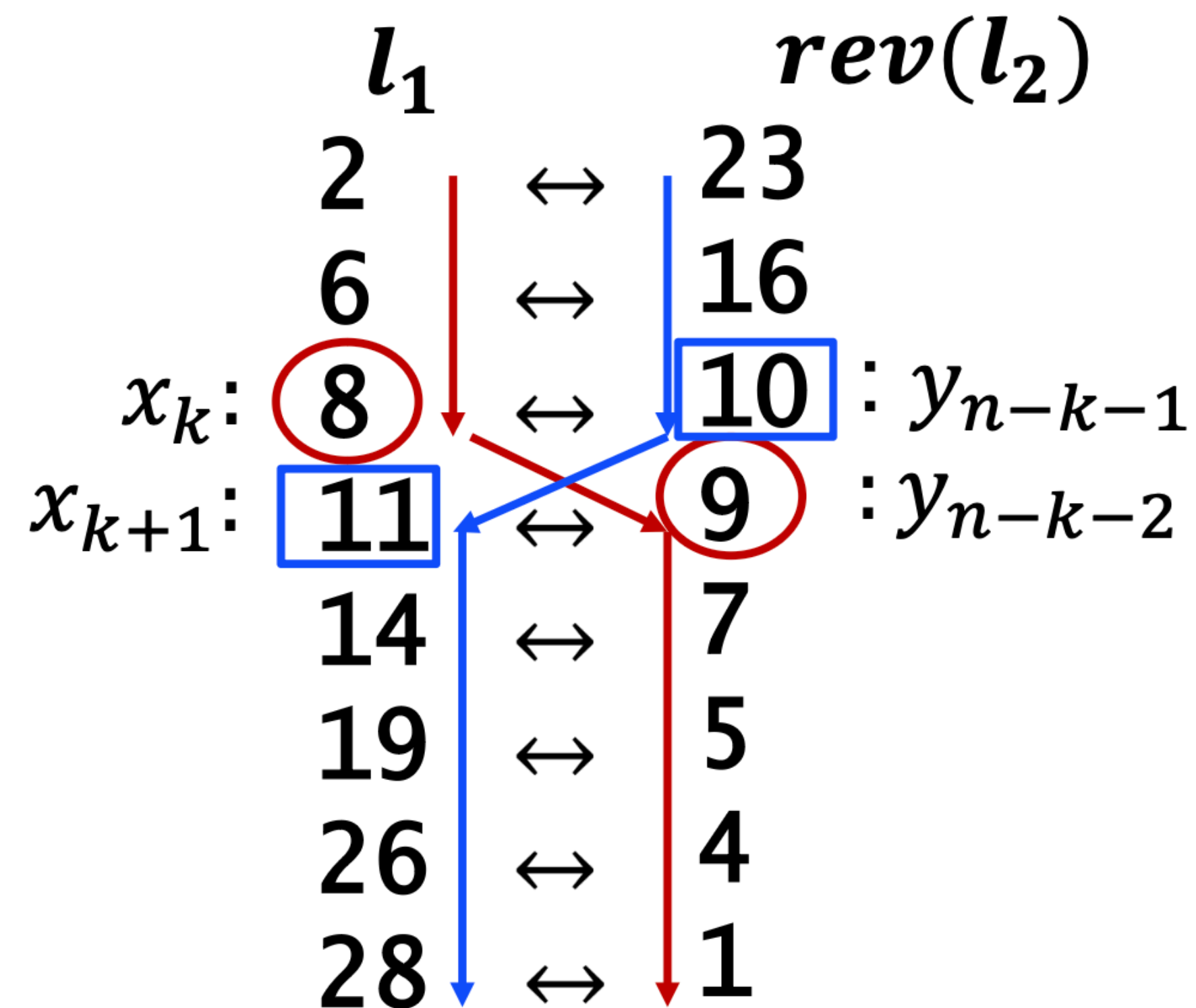
Example: Bitonic Split

- $l_1: x_0, x_1, \dots, x_{n-1}$
- $l_2: y_0, y_1, \dots, y_{n-1}$



Proof: Bitonic Split

- Prove: $\max(l_{\min}) \leq \min(l_{\max})$
- $\max(l_{\min}) = \max(x_k, y_{n-k-2})$
- $\min(l_{\max}) = \min(x_{k+1}, y_{n-k-1})$
- Proof:
 1. $x_k \leq x_{k+1}$
 - l_1 is sorted
 2. $x_k \leq y_{n-k-1}$
 - Transition after k
 3. $y_{n-k-2} \leq x_{k+1}$
 - Transition after k
 4. $y_{n-k-2} \leq y_{n-k-1}$
 - l_2 is sorted



First Attempt: Bitonic Sort Analysis

Recursive sort

Pairwise comparison

- $T(n, p) = T\left(\frac{n}{2}, \frac{p}{2}\right) + O\left(\frac{n}{p}\right) + T\left(\frac{n}{2}, \frac{p}{2}\right)$

- $T(n, p) = 2T\left(\frac{n}{2}, \frac{p}{2}\right) + O\left(\frac{n}{p}\right)$

Sort lmin and lmax

- $T(n, p) = 2\left(2T\left(\frac{n}{4}, \frac{p}{4}\right) + O\left(\frac{n/2}{p/2}\right)\right) + O\left(\frac{n}{p}\right)$

- $T(n, p) = 2^2T\left(\frac{n}{4}, \frac{p}{4}\right) + O\left(2\frac{n}{p} + \frac{n}{p}\right)$

- $T(n, p) = 2^3T\left(\frac{n}{8}, \frac{p}{8}\right) + O\left(2^2\frac{n}{p} + 2\frac{n}{p} + \frac{n}{p}\right)$

- $T(n, p) = pT\left(\frac{n}{p}, 1\right) + O\left(\frac{n}{p}\left(1 + 2 + 2^2 + \dots + \frac{p}{2}\right)\right)$

$(p - 1)$

- Each processor sorts locally in $O\left(\frac{n}{p} \log \frac{n}{p}\right)$ time.

- $T(n, p) = O\left(\cancel{p} \frac{n}{\cancel{p}} \log \frac{n}{p} + n\right) = O\left(n \log \frac{n}{p}\right)$

First Attempt: Bitonic Sort Analysis

Recursive sort

Pairwise comparison

- $T(n, p) = T\left(\frac{n}{2}, \frac{p}{2}\right) + O\left(\frac{n}{p}\right) + T\left(\frac{n}{2}, \frac{p}{2}\right)$

- $T(n, p) = 2T\left(\frac{n}{2}, \frac{p}{2}\right) + O\left(\frac{n}{p}\right)$

Sort Imin and Imax

- $T(n, p) = 2\left(2T\left(\frac{n}{4}, \frac{p}{4}\right) + O\left(\frac{n/2}{p/2}\right)\right) + O\left(\frac{n}{p}\right)$

- $T(n, p) = 2^2T\left(\frac{n}{4}, \frac{p}{4}\right) + O\left(2\frac{n}{p} + \frac{n}{p}\right)$

- $T(n, p) = 2^3T\left(\frac{n}{8}, \frac{p}{8}\right) + O\left(2^2\frac{n}{p} + 2\frac{n}{p} + \frac{n}{p}\right)$

- $T(n, p) = pT\left(\frac{n}{p}, 1\right) + O\left(\frac{n}{p}\left(1 + 2 + 2^2 + \dots + 2^{p-1}\right)\right)$

$(p - 1)$

- Each processor sorts locally in $O\left(\frac{n}{p} \log \frac{n}{p}\right)$ time.

- $T(n, p) = O\left(\cancel{p} \frac{n}{\cancel{p}} \log \frac{n}{p} + n\right)$

$$= O\left(n \log \frac{n}{p}\right)$$

Even worse than merge sort with serial merge, which was $O((n/p) \lg(n/p))$

First Attempt: Bitonic Sort Analysis

Recursive sort

Pairwise comparison

Problem -
exponentially
increasing down the
recursion - 4 total
sorts and 2 can
happen in parallel at
once

- $T(n, p) = T\left(\frac{n}{2}, \frac{p}{2}\right) + O\left(\frac{n}{p}\right) + T\left(\frac{n}{2}, \frac{p}{2}\right)$

Sort lmin and lmax

- $2T\left(\frac{n}{2}, \frac{p}{2}\right) + O\left(\frac{n}{p}\right)$

- $T(n, p) = 2\left(2T\left(\frac{n}{4}, \frac{p}{4}\right) + O\left(\frac{n/2}{p/2}\right)\right) + O\left(\frac{n}{p}\right)$

- $T(n, p) = 2^2 T\left(\frac{n}{4}, \frac{p}{4}\right) + O\left(2\frac{n}{p} + \frac{n}{p}\right)$

- $T(n, p) = 2^3 T\left(\frac{n}{8}, \frac{p}{8}\right) + O\left(2^2 \frac{n}{p} + 2\frac{n}{p} + \frac{n}{p}\right)$

- $T(n, p) = pT\left(\frac{n}{p}, 1\right) + O\left(\frac{n}{p}\left(1 + 2 + 2^2 + \dots + 2^{p-1}\right)\right)$

$(p - 1)$

- Each processor sorts locally in $O\left(\frac{n}{p} \log \frac{n}{p}\right)$ time.

- $T(n, p) = O\left(\cancel{p} \frac{n}{\cancel{p}} \log \frac{n}{p} + n\right)$

$$= O\left(n \log \frac{n}{p}\right)$$

Even worse than merge
sort with serial merge,
which was
 $O((n/p) \lg(n/p))$

First Attempt: Bitonic Sort Analysis

Recursive sort

Pairwise comparison

Problem - exponentially increasing down the recursion - 4 total sorts and 2 can happen in parallel at once

$$\bullet T(n, p) = T\left(\frac{n}{2}, \frac{p}{2}\right) + O\left(\frac{n}{p}\right) + T\left(\frac{n}{2}, \frac{p}{2}\right)$$

Sort Imin and Imax

$$\bullet 2T\left(\frac{n}{2}, \frac{p}{2}\right) + O\left(\frac{n}{p}\right)$$

$$\bullet T(n, p) = 2\left(2T\left(\frac{n}{4}, \frac{p}{4}\right) + O\left(\frac{n/2}{p/2}\right)\right) + O\left(\frac{n}{p}\right)$$

$$\bullet T(n, p) = 2^2 T\left(\frac{n}{4}, \frac{p}{4}\right) + O\left(2\frac{n}{p} + \frac{n}{p}\right)$$

$$\bullet T(n, p) = 2^3 T\left(\frac{n}{8}, \frac{p}{8}\right) + O\left(2^2 \frac{n}{p} + 2\frac{n}{p} + \frac{n}{p}\right)$$

$$\bullet T(n, p) = pT\left(\frac{n}{p}, 1\right) + O\left(\frac{n}{p}\left(1 + 2 + 2^2 + \dots + 2^{p-1}\right)\right)$$

$$\bullet \text{ Each processor sorts locally in } O\left(\frac{n}{p} \log \frac{n}{p}\right) \text{ time.}$$

$$\bullet T(n, p) = O\left(\cancel{p} \frac{n}{\cancel{p}} \log \frac{n}{p} + n\right)$$

$$= O\left(n \log \frac{n}{p}\right)$$

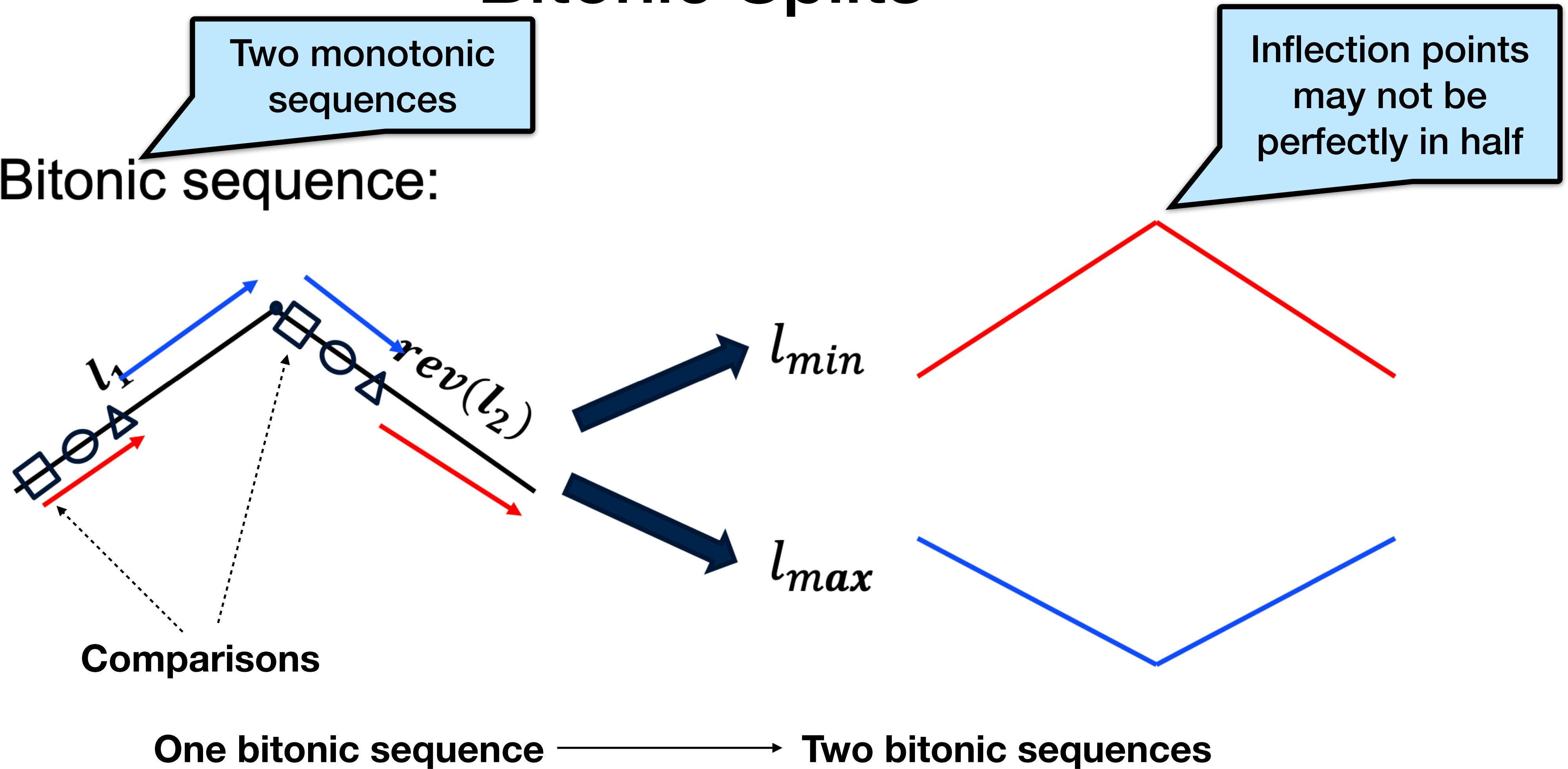
Observation: Imin and Imax are not arbitrary sequences. They are bitonic - can we use this?

(p - 1)

Even worse than merge sort with serial merge, which was $O((n/p) \lg(n/p))$

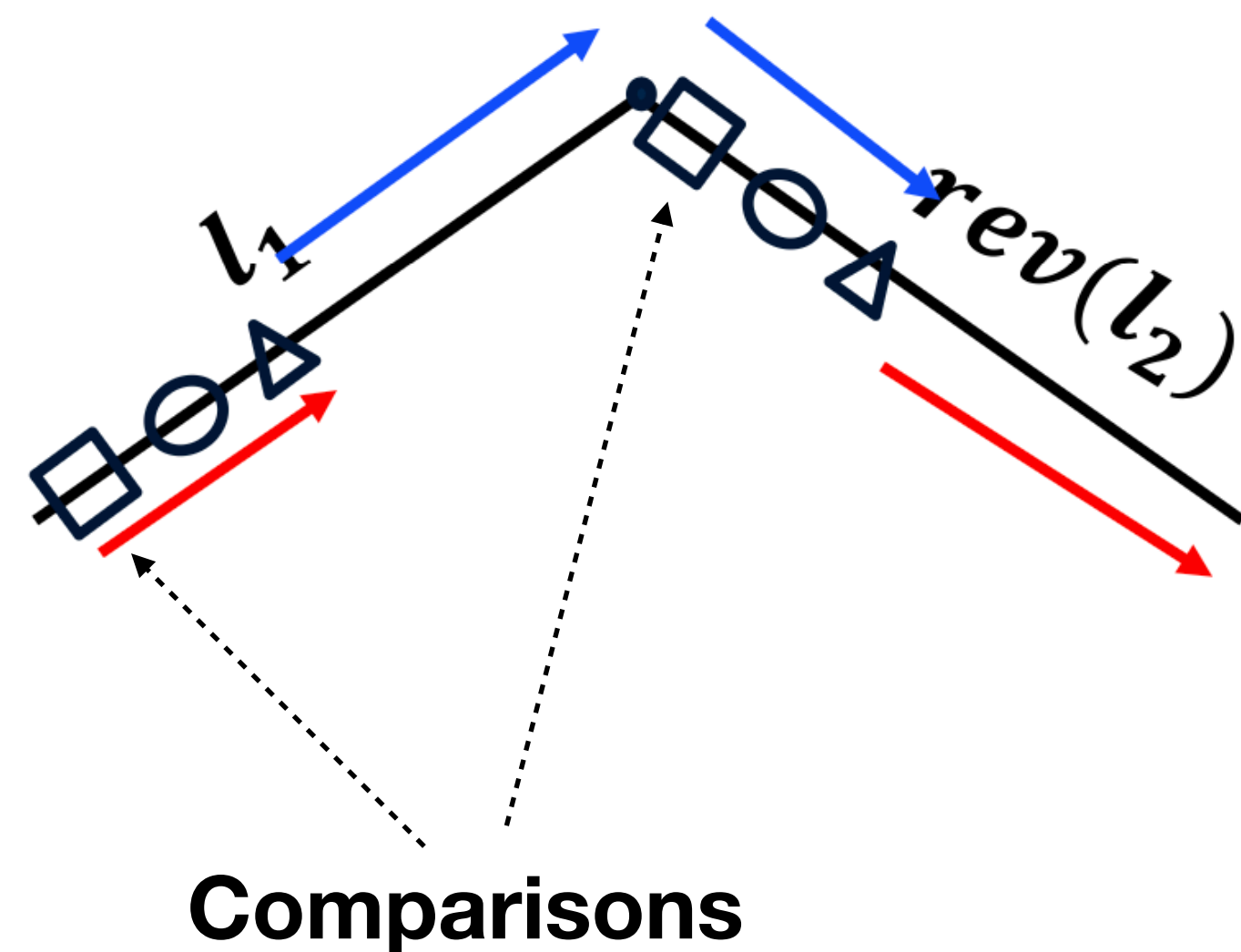
Bitonic Splits

- Bitonic sequence:



Bitonic Splits

- Bitonic sequence:



One bitonic sequence

l_{min}

l_{max}

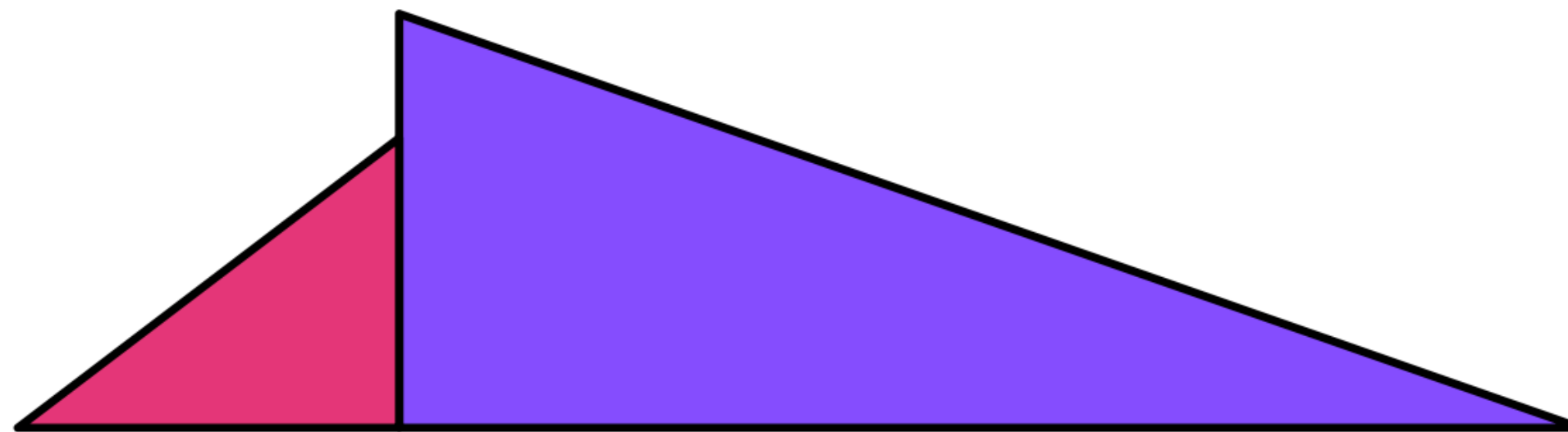
Two bitonic sequences

Inflection points
may not be
perfectly in half

In this case - is it
true in general?

Goal: Apply repeated bitonic splits with no additional work
(e.g., do not figure out where the inflection points are)

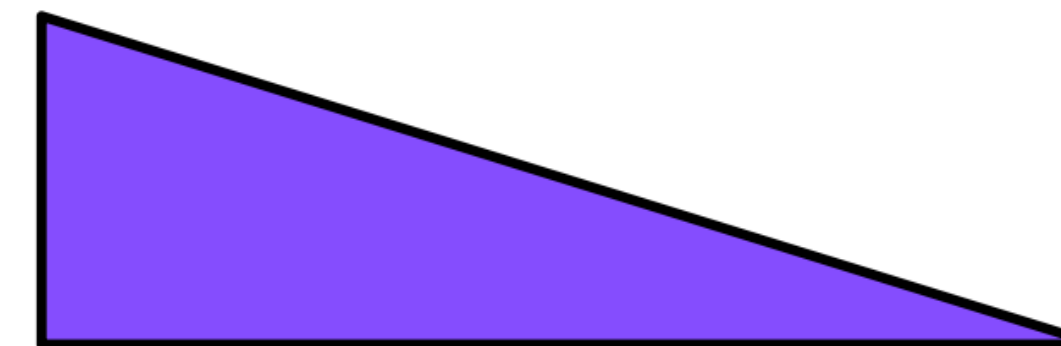
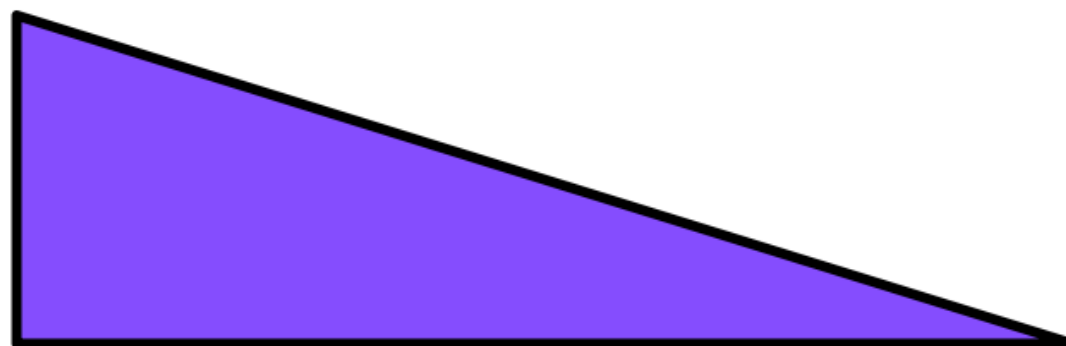
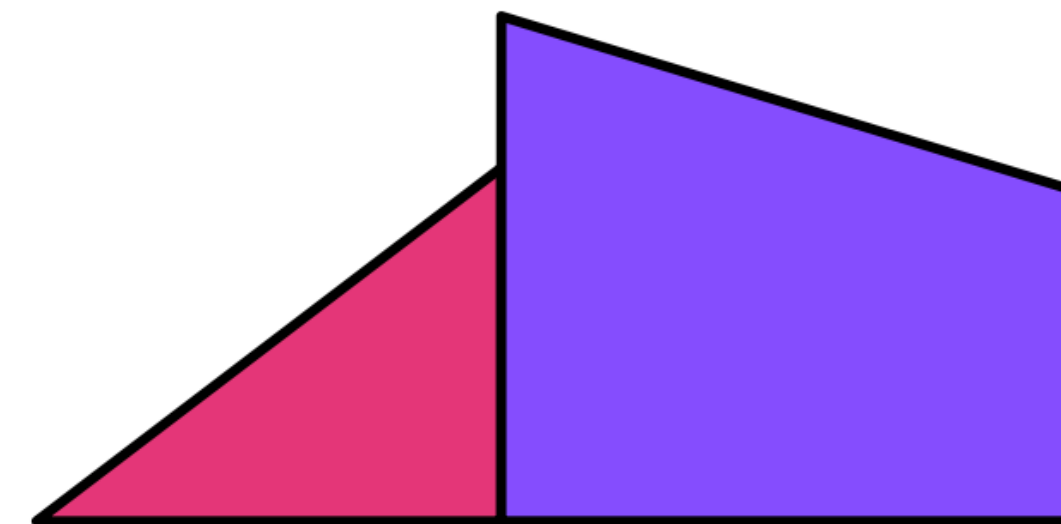
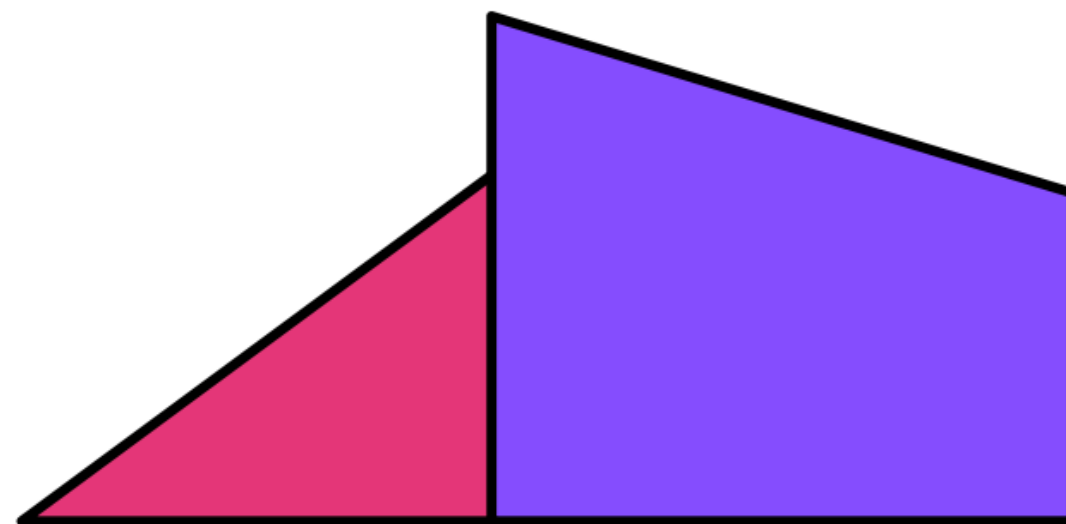
Splitting Bitonic Sequences



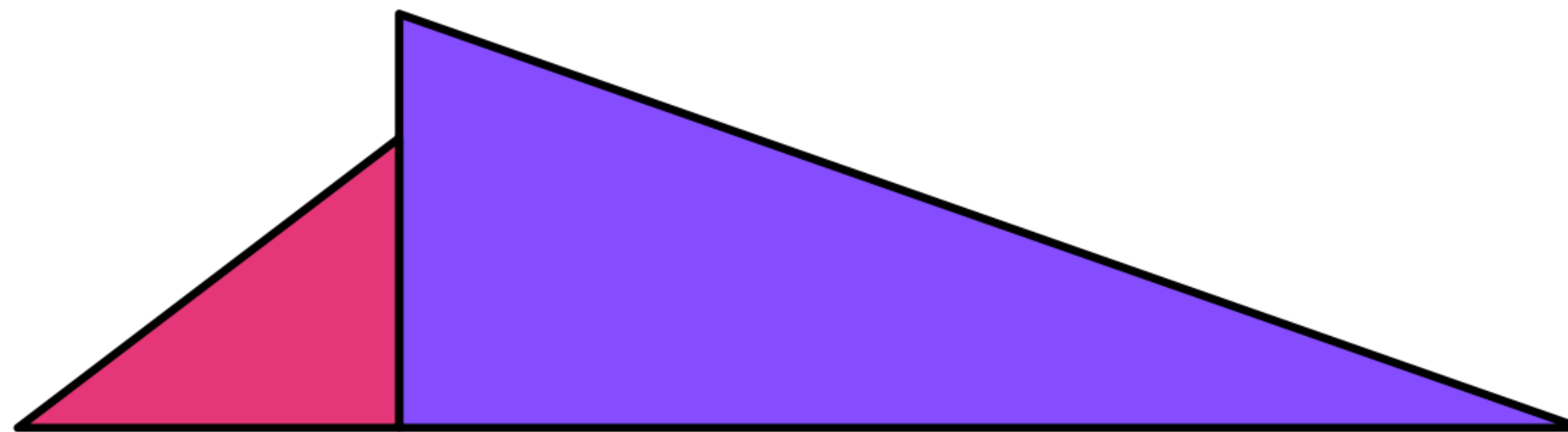
Sequence properties

s_1 and s_2 are both bitonic

$\forall_x \forall_y x \in s_1, y \in s_2, x < y$



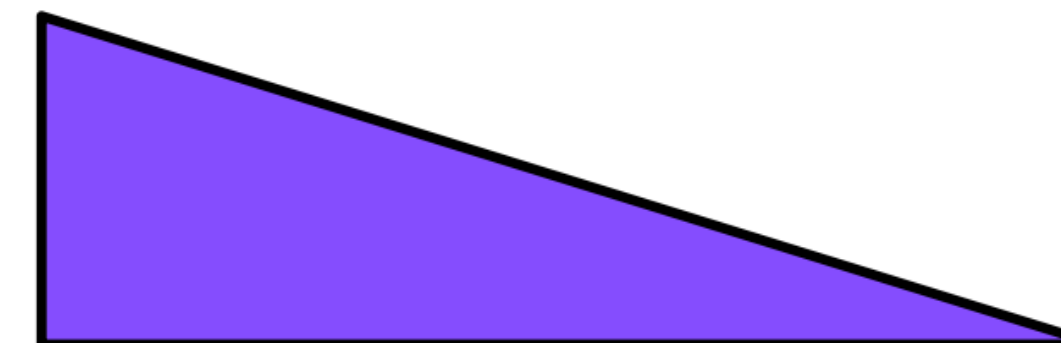
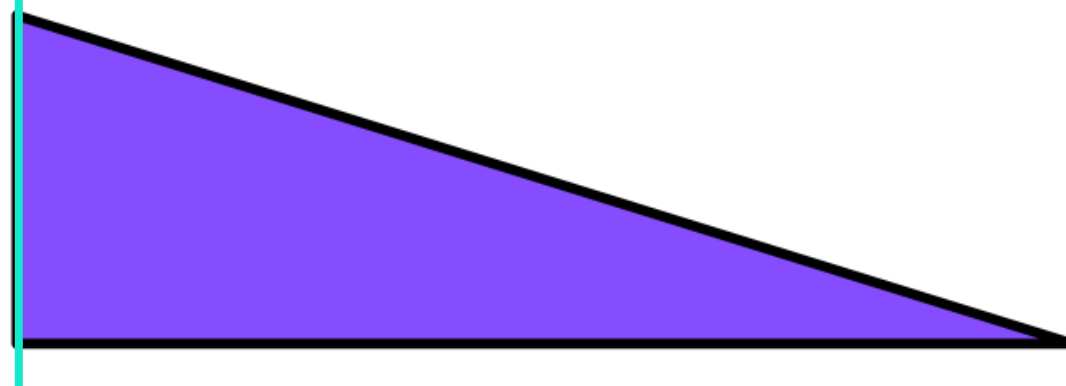
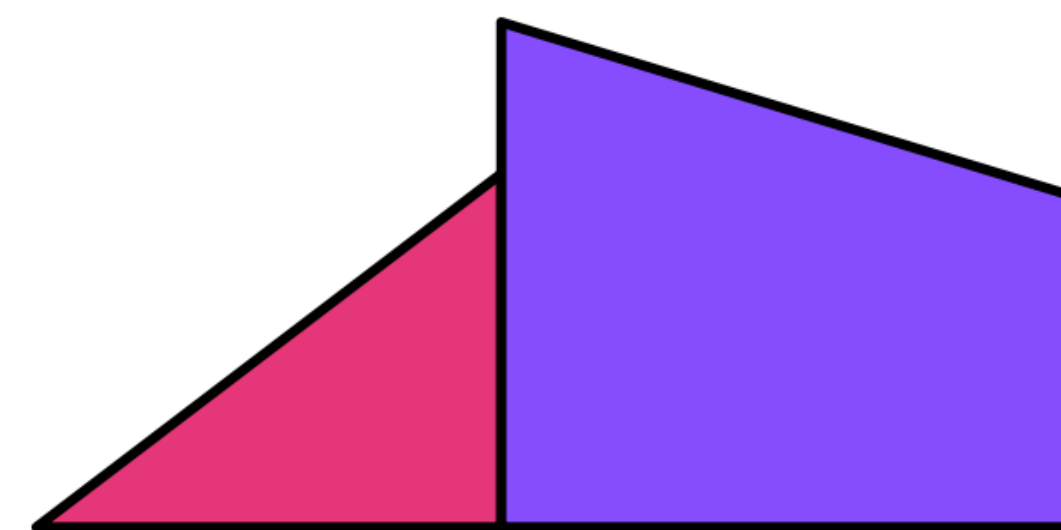
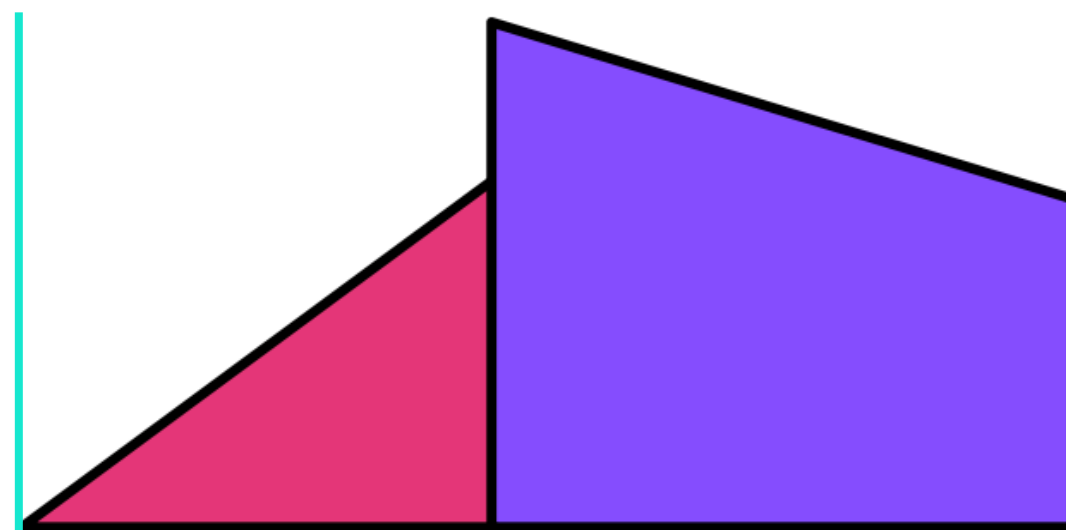
Splitting Bitonic Sequences



Sequence properties

s_1 and s_2 are both bitonic

$\forall x \forall y x \in s_1, y \in s_2, x < y$

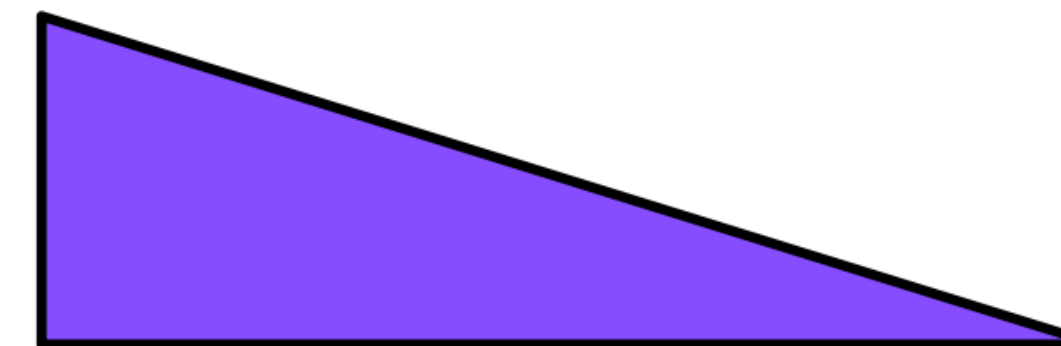
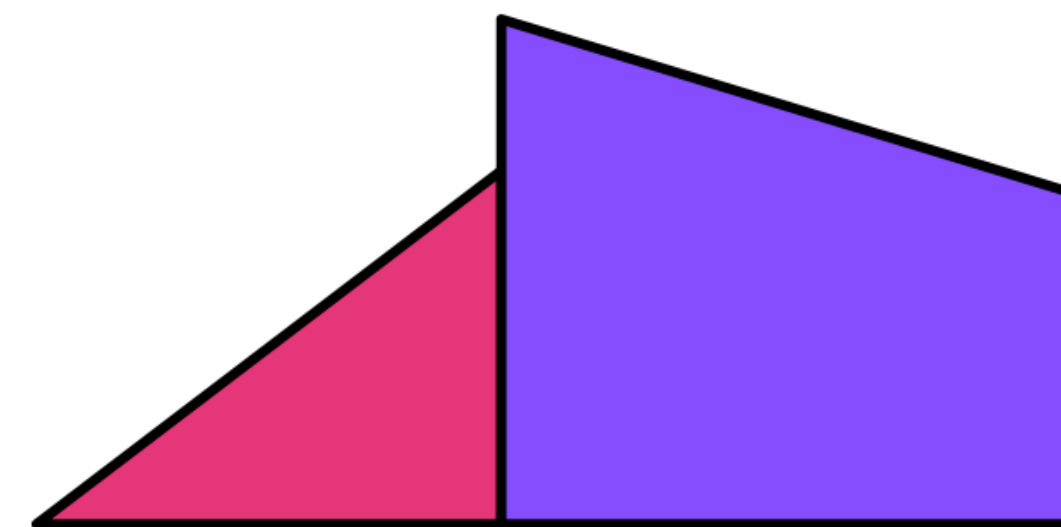
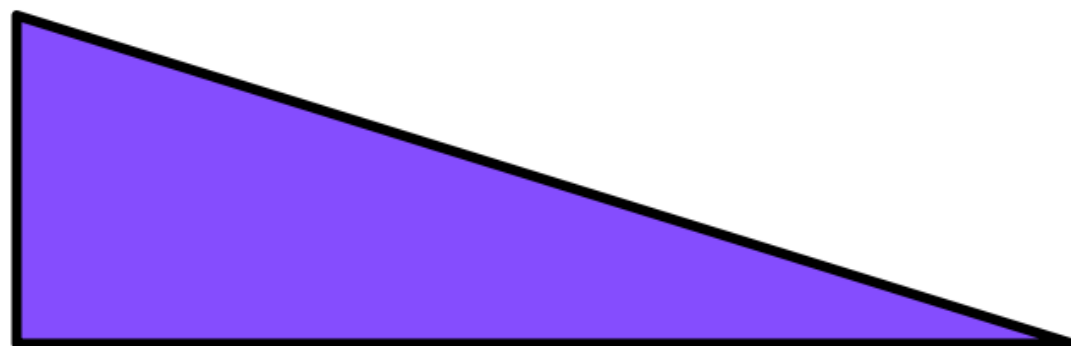
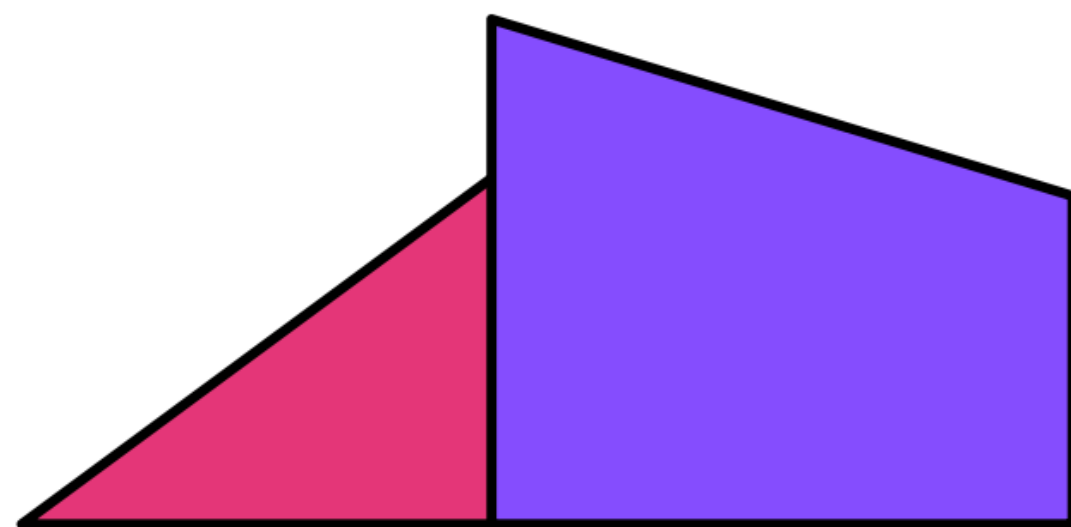
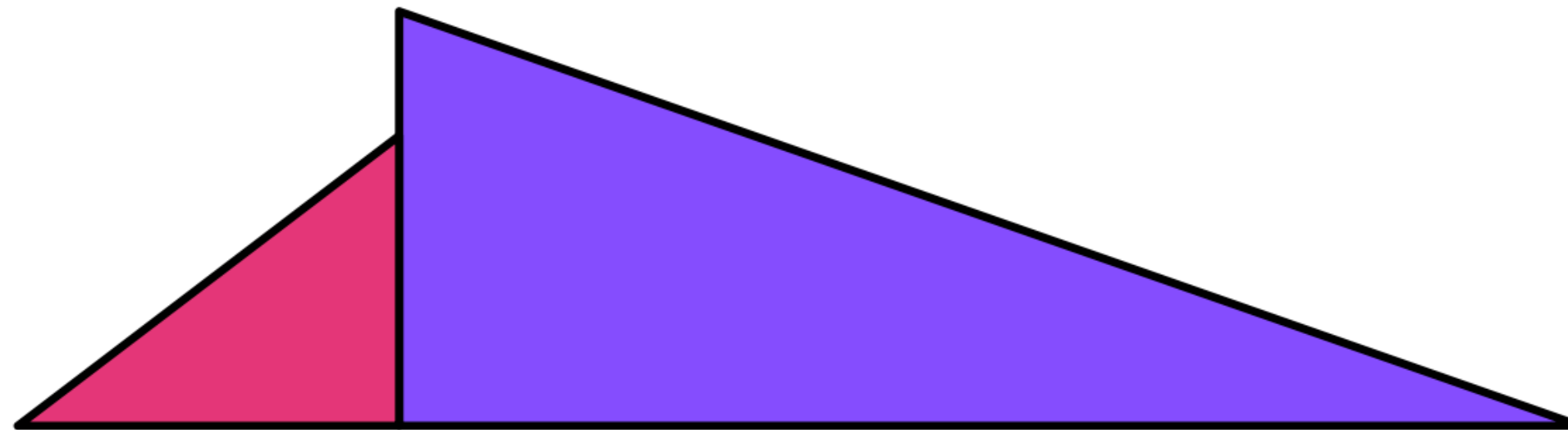


Splitting Bitonic Sequences

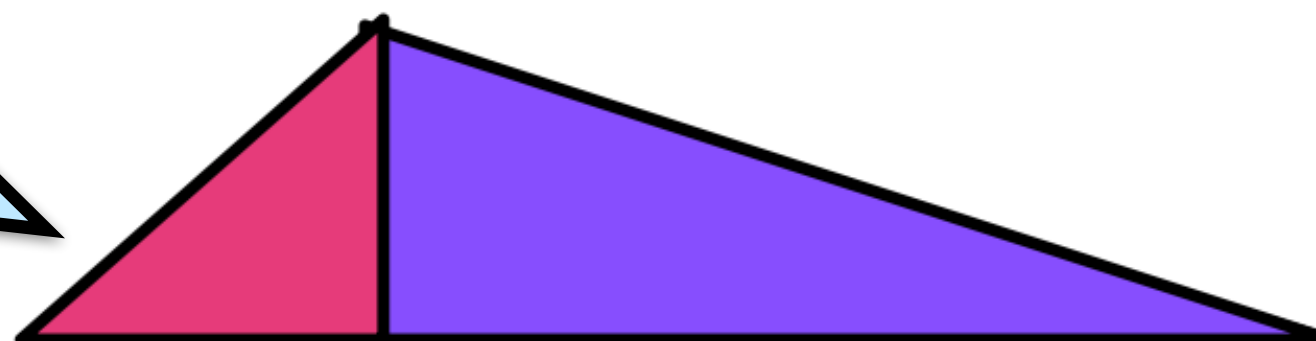
Sequence properties

s_1 and s_2 are both bitonic

$\forall x \forall y x \in s_1, y \in s_2, x < y$



Bitonic!



min

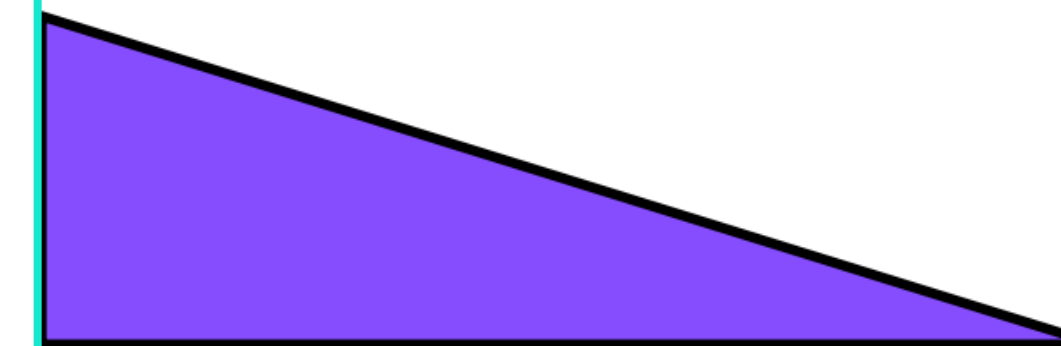
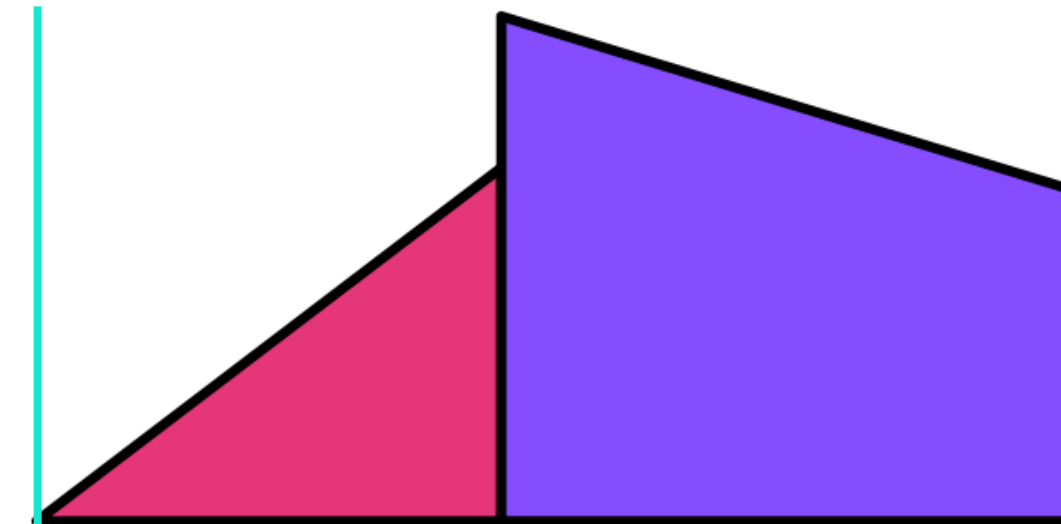
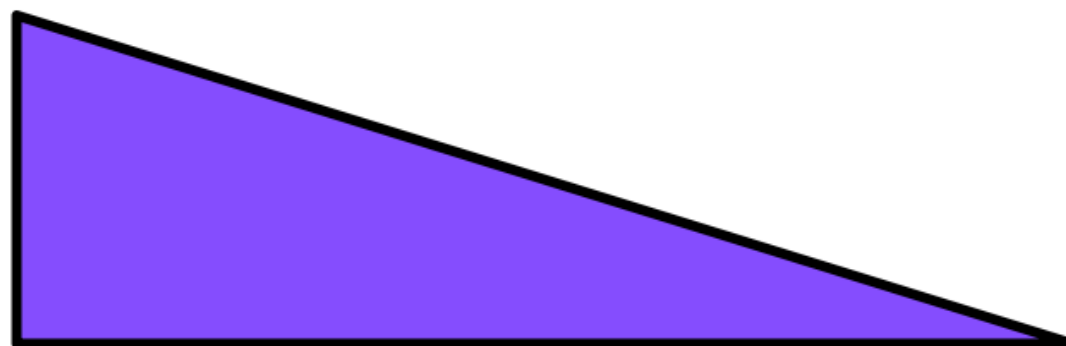
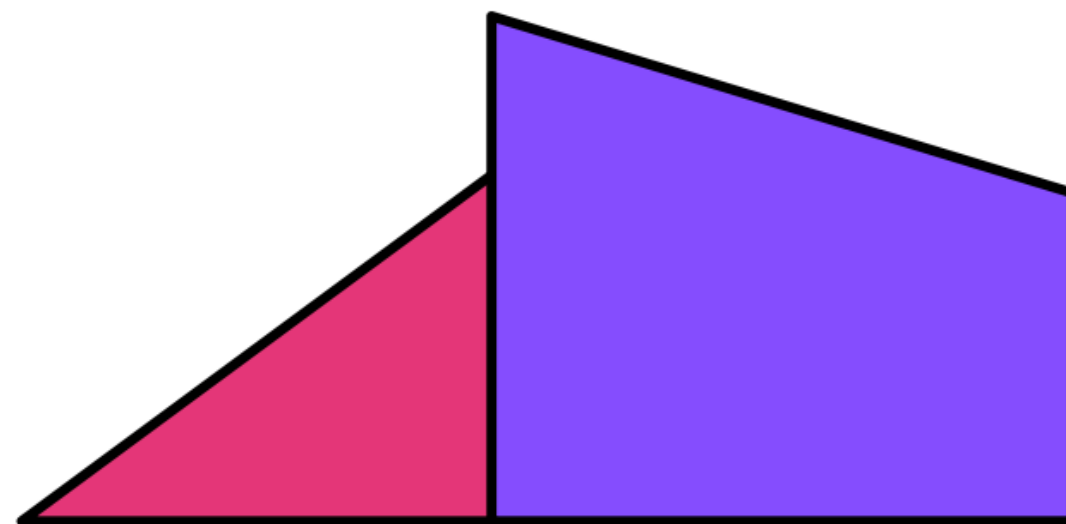
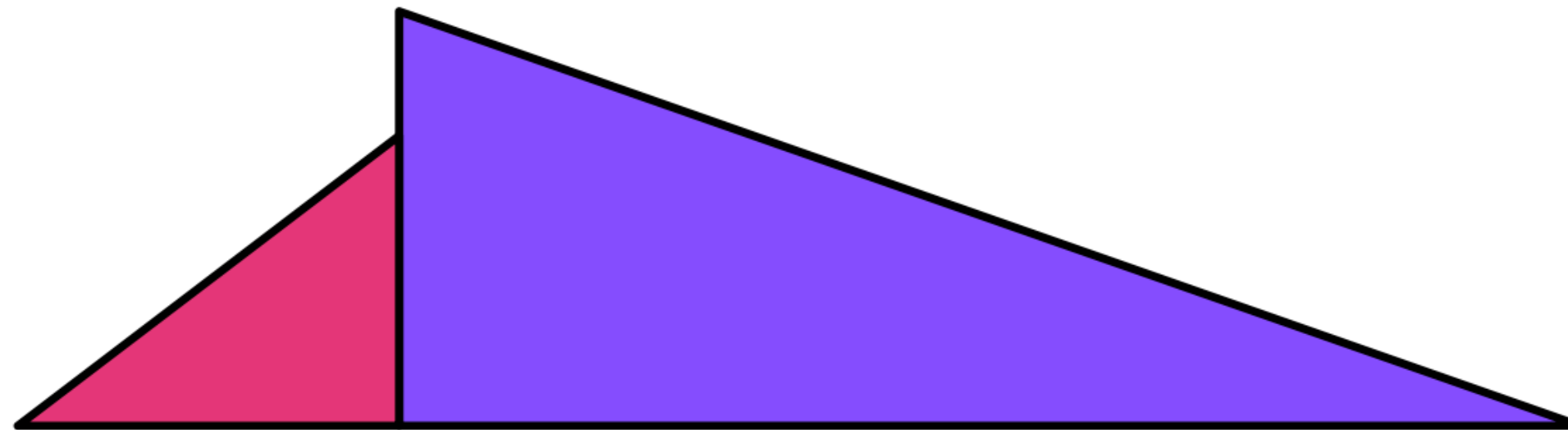
max

Splitting Bitonic Sequences

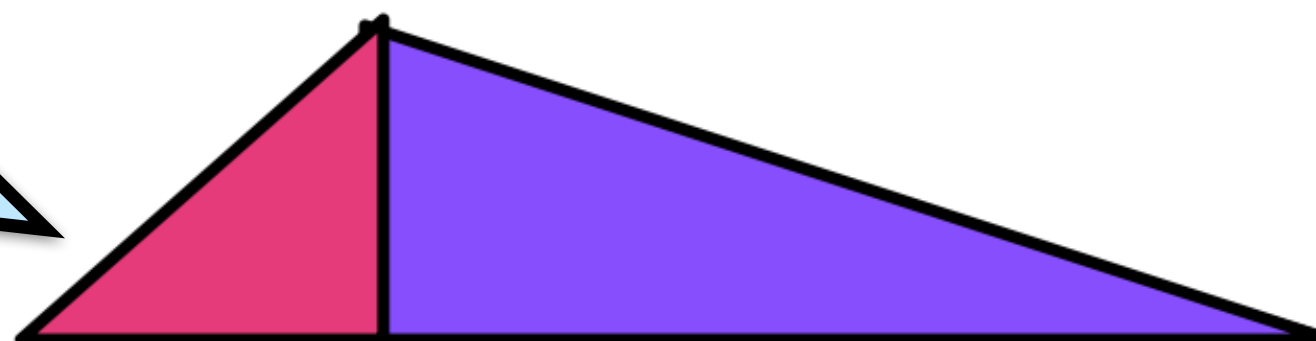
Sequence properties

s_1 and s_2 are both bitonic

$\forall x \forall y x \in s_1, y \in s_2, x < y$



Bitonic!



min

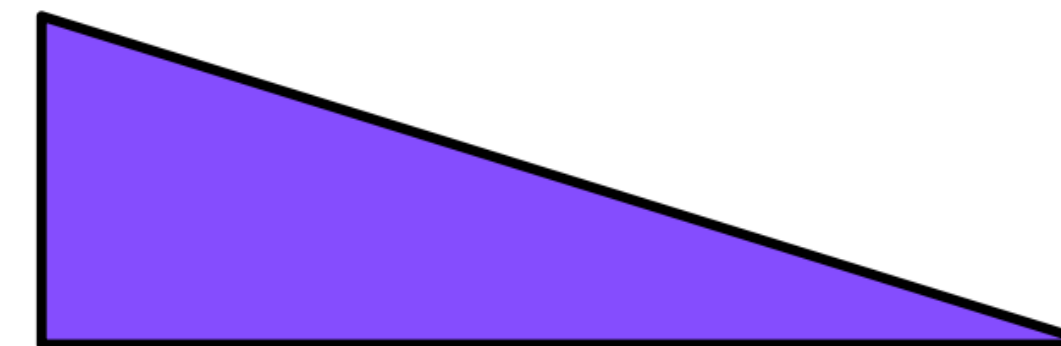
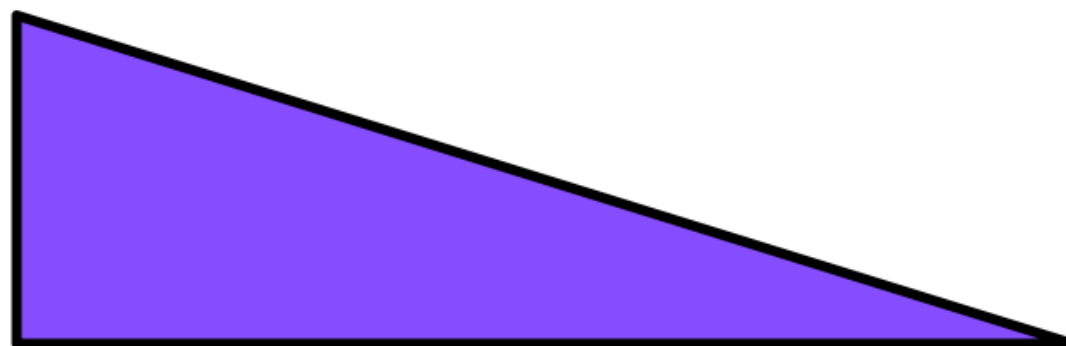
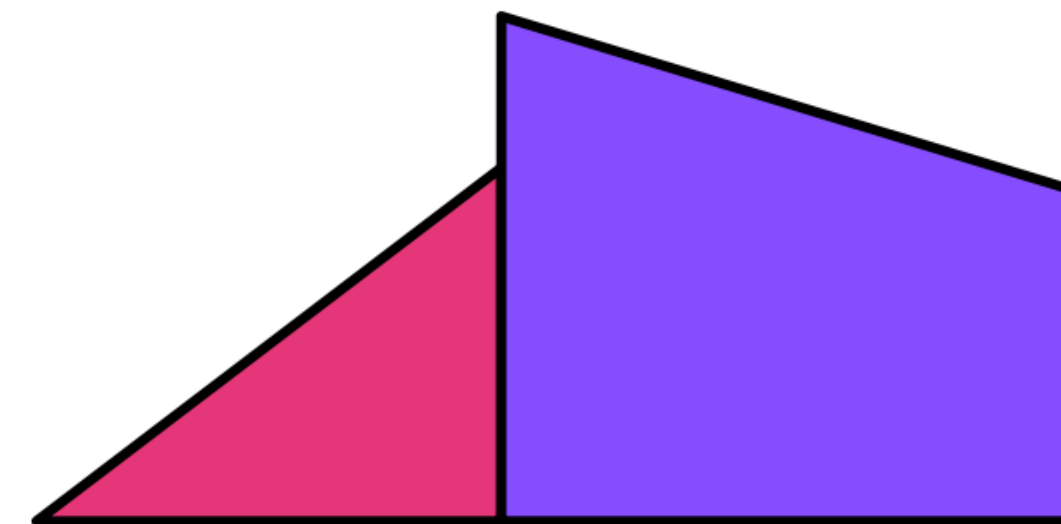
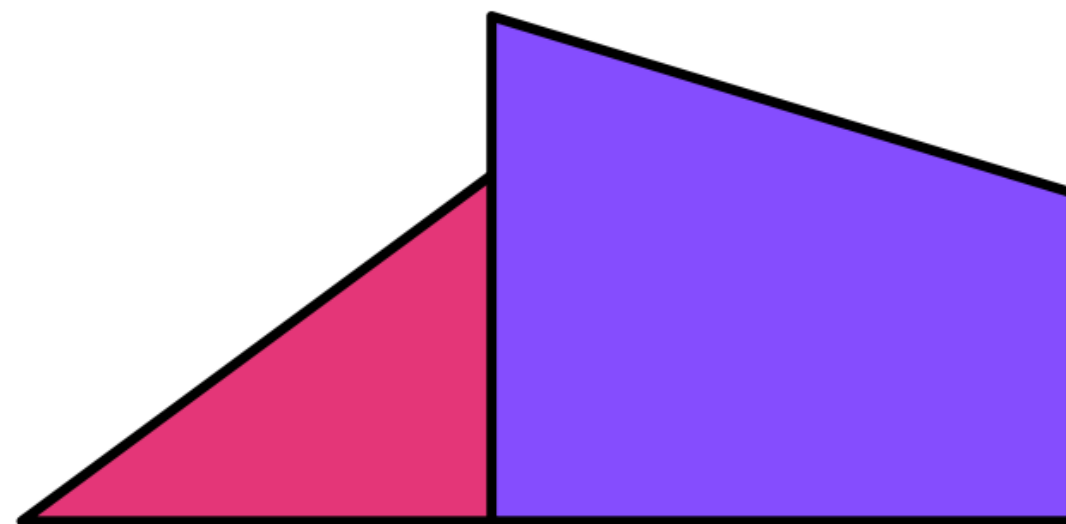
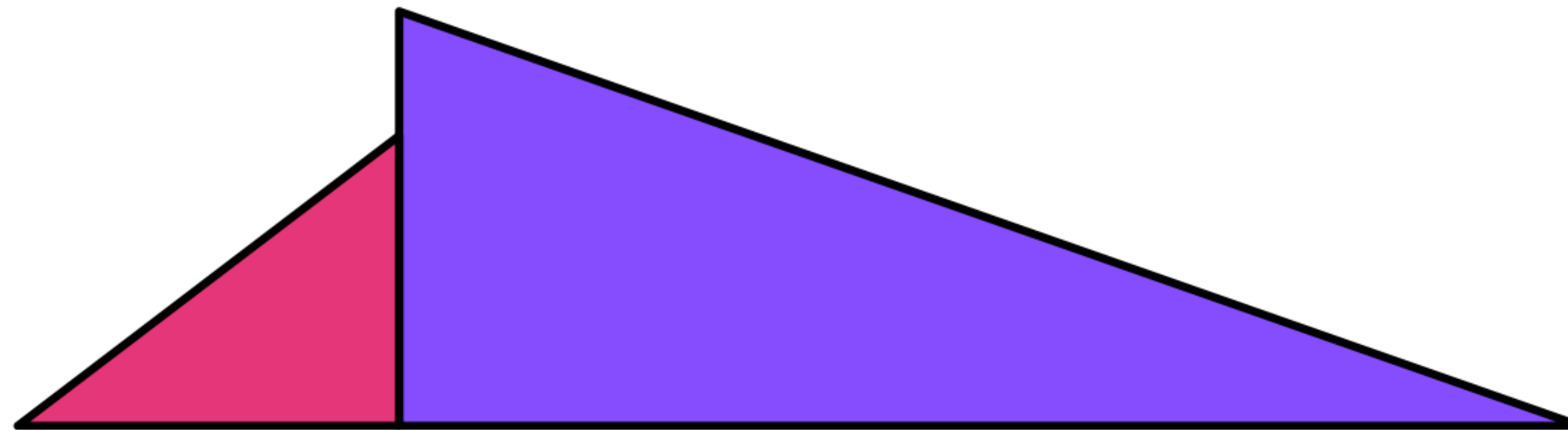
max

Splitting Bitonic Sequences

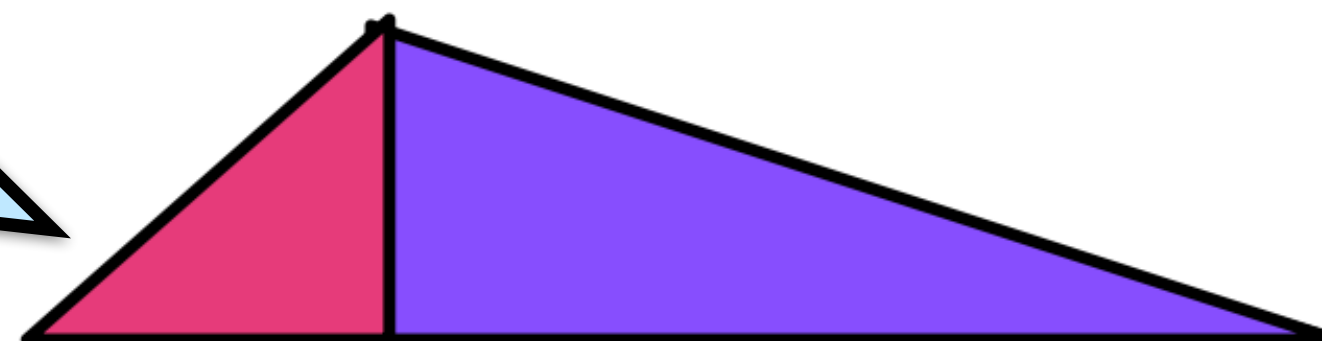
Sequence properties

s_1 and s_2 are both bitonic

$\forall_x \forall_y x \in s_1, y \in s_2, x < y$

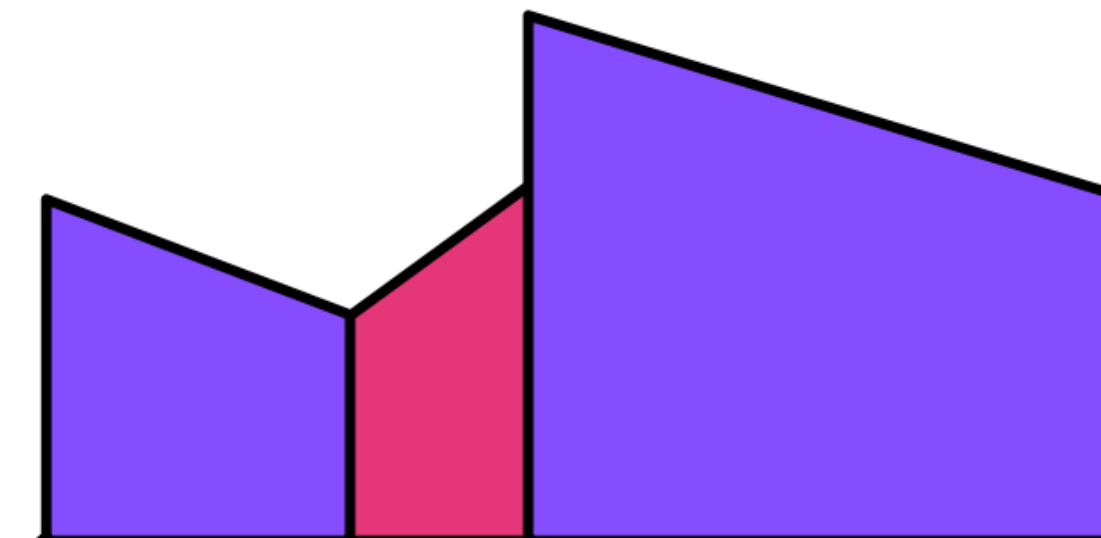


Bitonic!



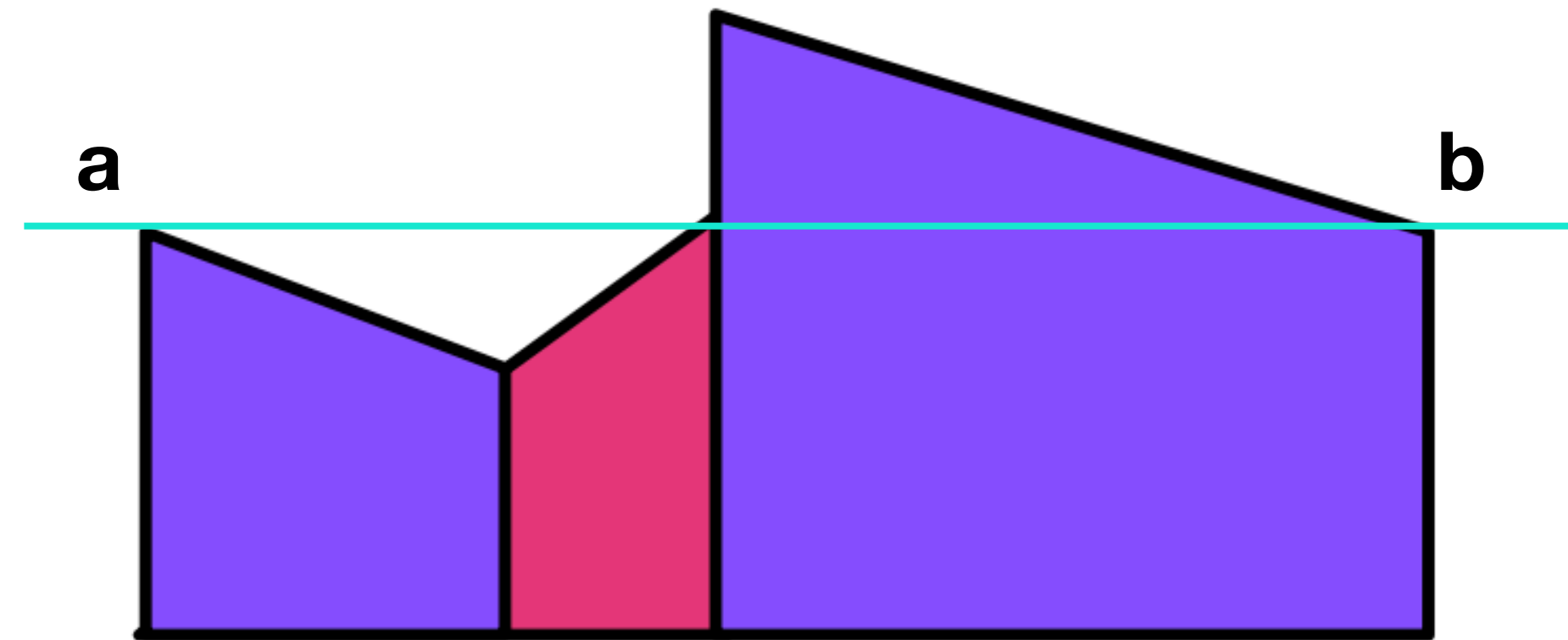
min

Not bitonic?



max

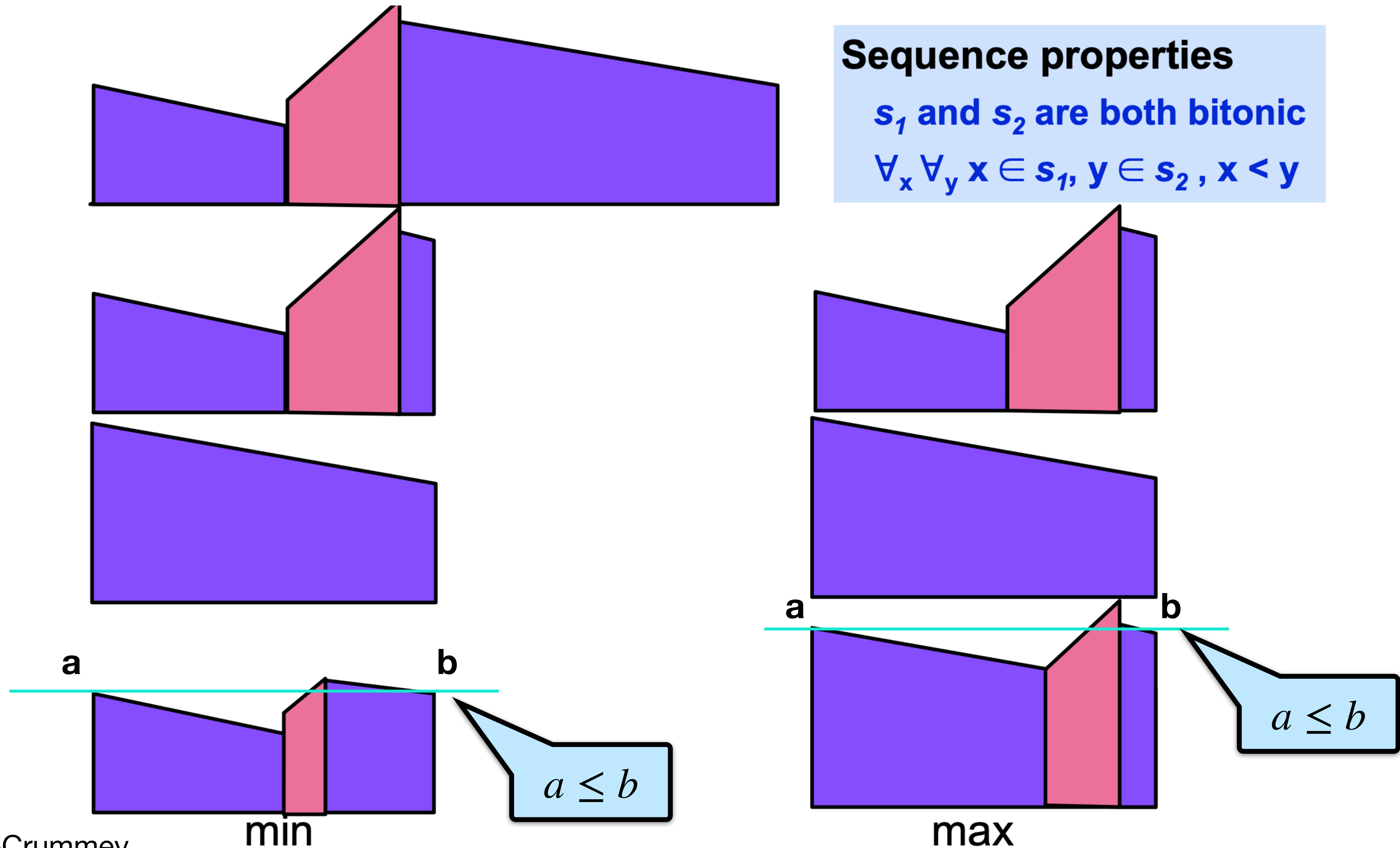
Observation: Properties of l_{max}



- By construction, $a \leq b$ because the part we got them from was monotonically decreasing.
- So l_{max} is a **cyclical shift** of a bitonic sequence.

To be clear, we do not want to actually find where the rotation point is

Splitting Bitonic Sequences

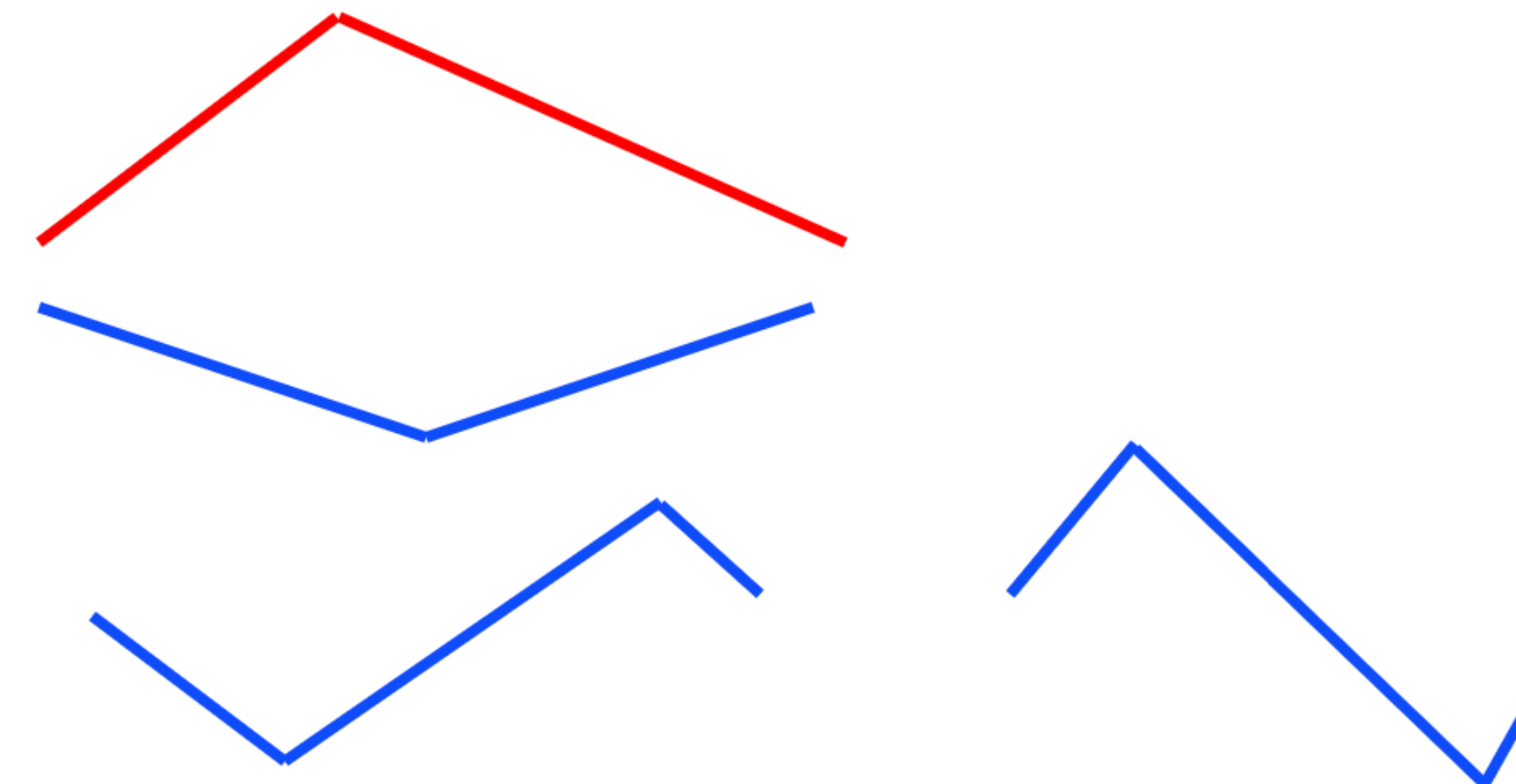


Bitonic Sequence - Formal Definition

- **Bitonic sequence:** $x_0, x_1, \dots, x_{n-1}$ is bitonic if $\exists k$ such that
 - **(T1)** $x_0, x_1, \dots, x_k$ is non-decreasing and $x_{k+1}, x_{k+2}, \dots, x_{n-1}$ is non-increasing OR
 - **(T2)** $x_0, x_1, \dots, x_k$ is non-increasing and $x_{k+1}, x_{k+2}, \dots, x_{n-1}$ is non-decreasing OR
 - **(T3)** There is a cyclical shift of the sequence that makes (a) or (b) true.

- **Examples:**

- 3, 5, 7, 10, 15, 14, 12, 9, 8, 4, 1
- 14, 12, 9, 8, 4, 1, 3, 5, 7, 10, 15
- 8, 4, 1, 3, 5, 7, 10, 15, 14, 12, 9



Bitonic Split

- **Bitonic split:** of a bitonic sequence $l = x_0, x_1, \dots, x_{n-1}$ is defined as decomposition of l into
- $l_{min} = \min(x_0, x_{\frac{n}{2}}), \min(x_1, x_{\frac{n}{2}+1}), \dots, \min(x_{\frac{n}{2}-1}, x_{n-1})$
- $l_{max} = \max(x_0, x_{\frac{n}{2}}), \max(x_1, x_{\frac{n}{2}+1}), \dots, \max(x_{\frac{n}{2}-1}, x_{n-1})$

Lemma

- Let l be a bitonic sequence, and l_{min} and l_{max} result from its bitonic split, then

1. l_{min} and l_{max} are bitonic
2. $\max(l_{min}) \leq \min(l_{max})$

for recursion

avoid sorting

Algorithm - given the two sorted lists l_1, l_2 , avoid sorting and just keep applying bitonic split

Lemma

- Let l be a bitonic sequence, and l_{min} and l_{max} result from its bitonic split, then

- l_{min} and l_{max} are bitonic
- $\max(l_{min}) \leq \min(l_{max})$

for recursion

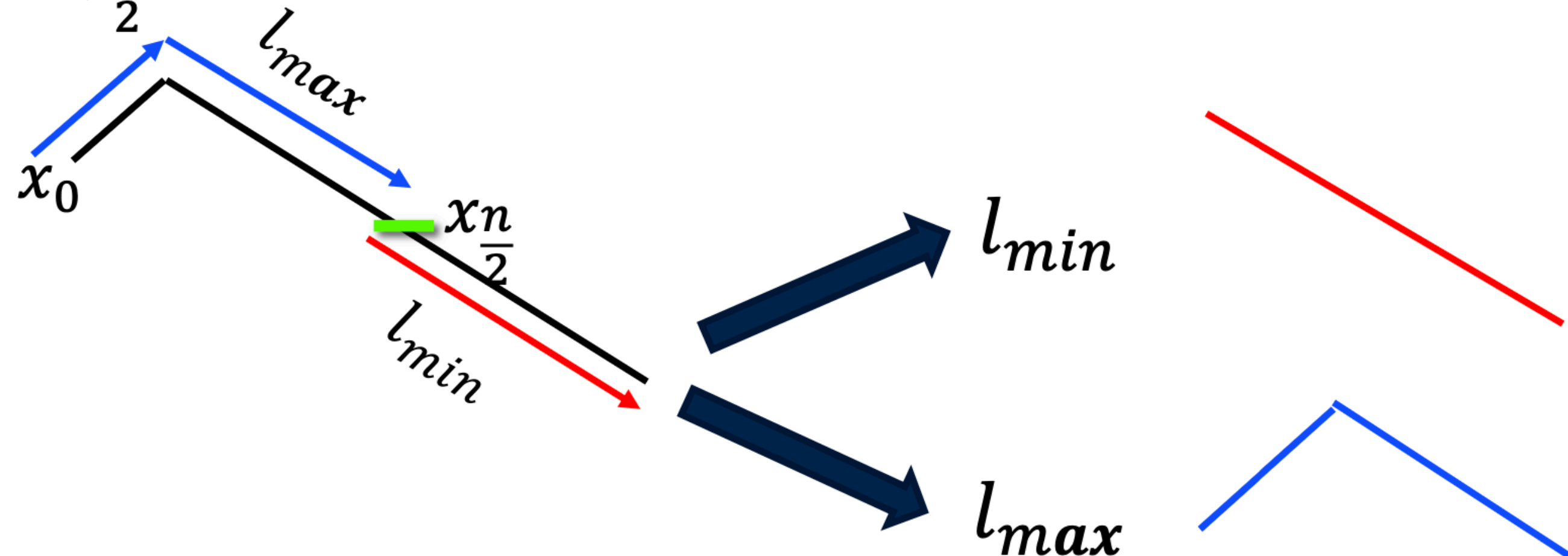
avoid sorting

Algorithm - given the two sorted lists l_1, l_2 , avoid sorting and just keep applying bitonic split

- Partial Proof:**

- Let l be a bitonic sequence of type T1

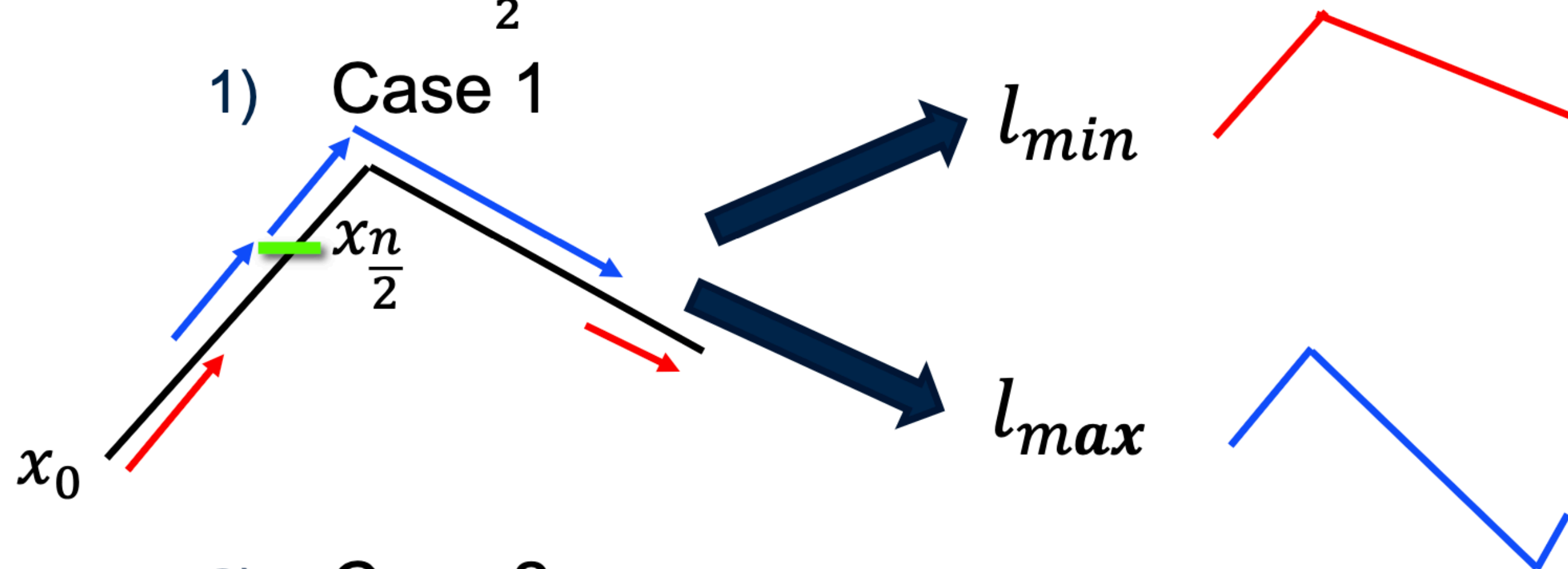
a) $x_0 > x_{\frac{n}{2}}$; Then, $k < \frac{n}{2}$



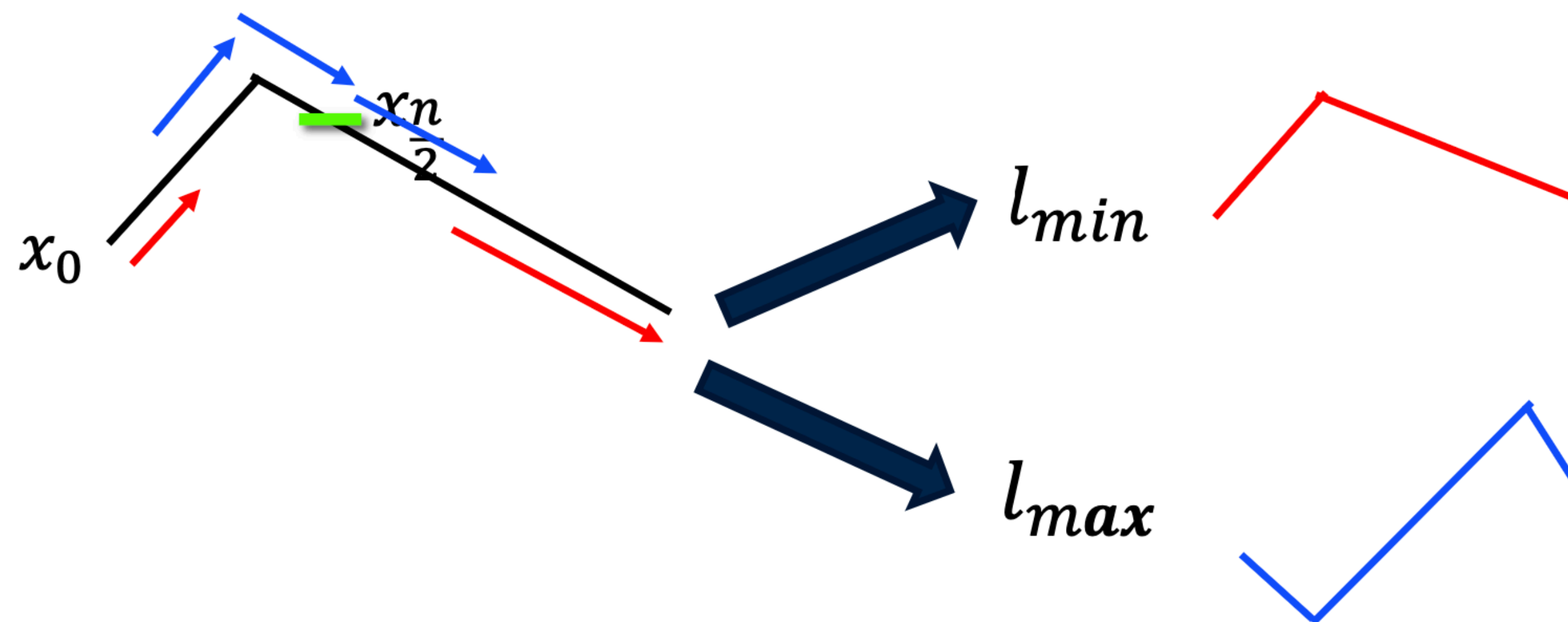
Partial proof of lemma

a) $x_0 < x_{\frac{n}{2}}$

1) Case 1



2) Case 2



Sub-Lemma

- Let l be a bitonic sequence and l' be a cyclical shift
- Bitonic split l into l_{min} and l_{max}
- Bitonic split l' into l'_{min} and l'_{max}
- Then, l'_{min} is a cyclical shift of l_{min} and
 l'_{max} is a cyclical shift of l_{max}

not actually doing the rotation,
just for analysis

Sub-Lemma

- Let l be a bitonic sequence and l' be a cyclical shift
 - Bitonic split l into l_{min} and l_{max}
 - Bitonic split l' into l'_{min} and l'_{max}
 - Then, l'_{min} is a cyclical shift of l_{min} and l'_{max} is a cyclical shift of l_{max}
- not actually doing the rotation, just for analysis
- Let
 - $l = x_0, x_1, x_2, \dots, x_{n-1}$ and
 - $l' = x_k, x_{k+1}, \dots, x_{n-1}, x_0, \dots, x_{k-1}$
 - $l_{min} = \min(x_0, x_{\frac{n}{2}}), \min(x_1, x_{\frac{n}{2}+1}), \dots, \min(x_{\frac{n}{2}-1}, x_{n-1})$
 - $l'_{min} = \min(x_k, x_{(k+\frac{n}{2}) \bmod n}), \dots$
 - Cyclical shift does not change the bitonic nature of the sequence nor the min (or max) element in the sequence!

Bitonic Merge Example

Sort a bitonic sequence through a sequence of bitonic splits

Example: use bitonic merge to sort 16-element bitonic sequence

How: perform a series of $\log_2 16 = 4$ bitonic splits

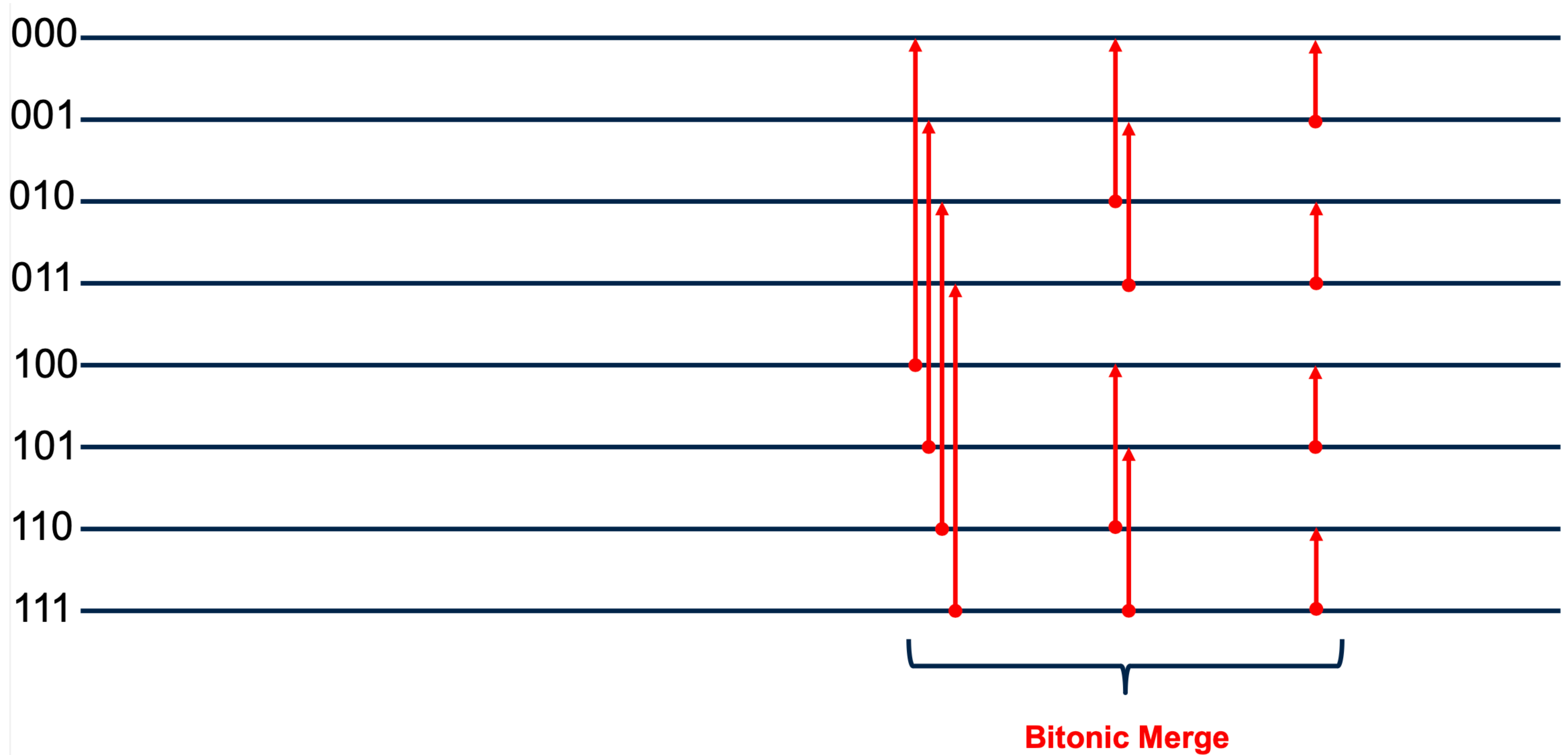
Original sequence	3	5	8	9	10	12	14	20	95	90	60	40	35	23	18	0
1st Split	3	5	8	9	10	12	14	0	95	90	60	40	35	23	18	20
2nd Split	3	5	8	0	10	12	14	9	35	23	18	20	95	90	60	40
3rd Split	3	0	8	5	10	9	14	12	18	20	35	23	60	40	95	90
4th Split	0	3	5	8	9	10	12	14	18	20	23	35	40	60	90	95

$$\text{Bitonic_merge}(p, p) = O(\log p)$$

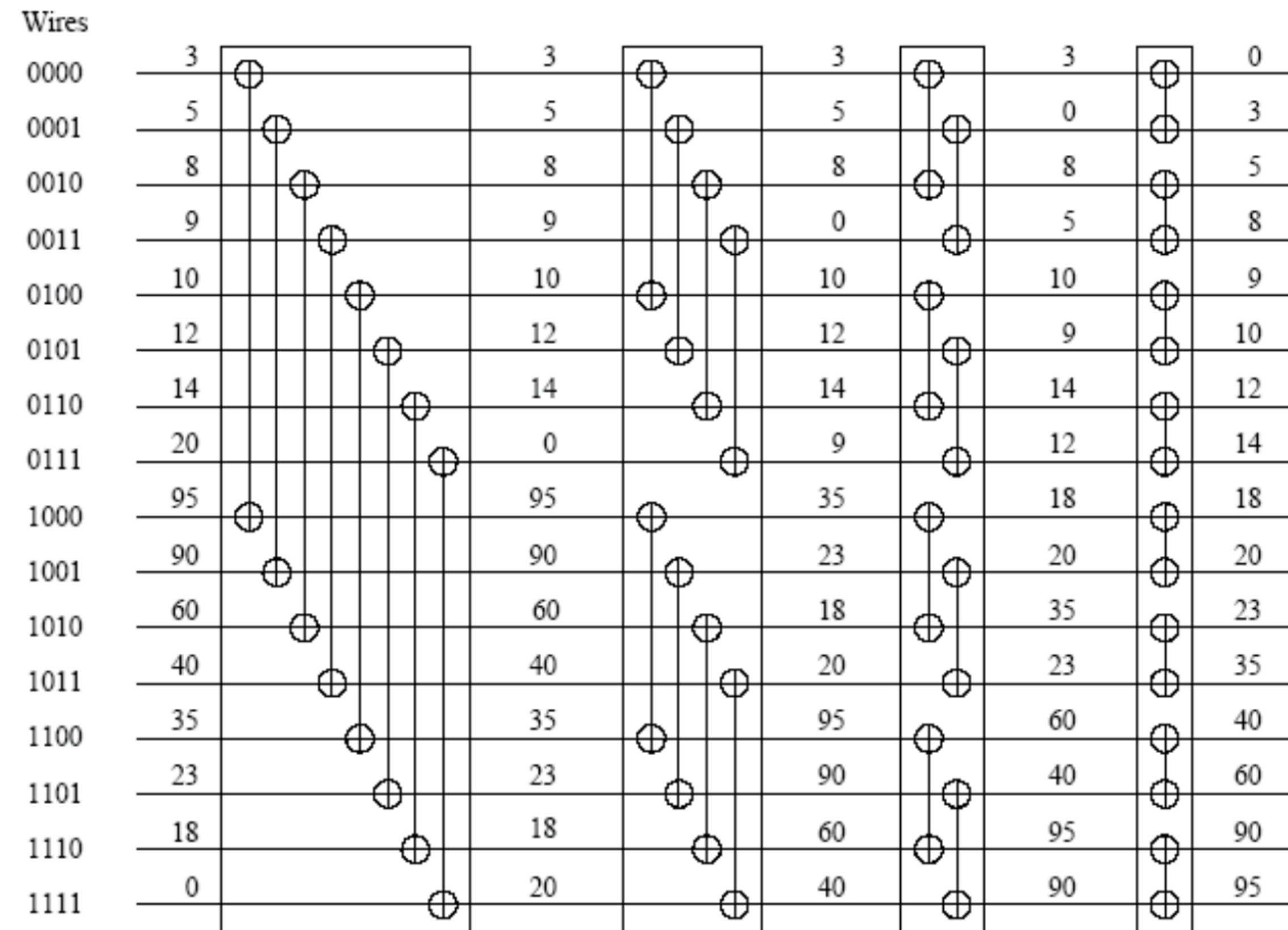
Sorting via Bitonic Merging Network

- **Sorting network can implement bitonic merge algorithm**
 - bitonic merging network*
- **Network structure**
 - $\log_2 n$ columns
 - each column
 - $n/2$ comparators
 - performs one step of the bitonic merge
- **Bitonic merging network with n inputs: $\oplus\text{BM}[n]$**
 - produces an increasing sequence
- **Replacing $\oplus$ comparators by $\ominus$ comparators: $\ominus\text{BM}[n]$**
 - produces a decreasing sequence

Bitonic Merge Example



Bitonic Merging Network



- **Input: bitonic sequence**
— input wires are numbered $0, 1, \dots, n - 1$ (shown in binary)
- **Output: sequence in sorted order**
- **Each column of comparators is drawn separately**

Bitonic Sort

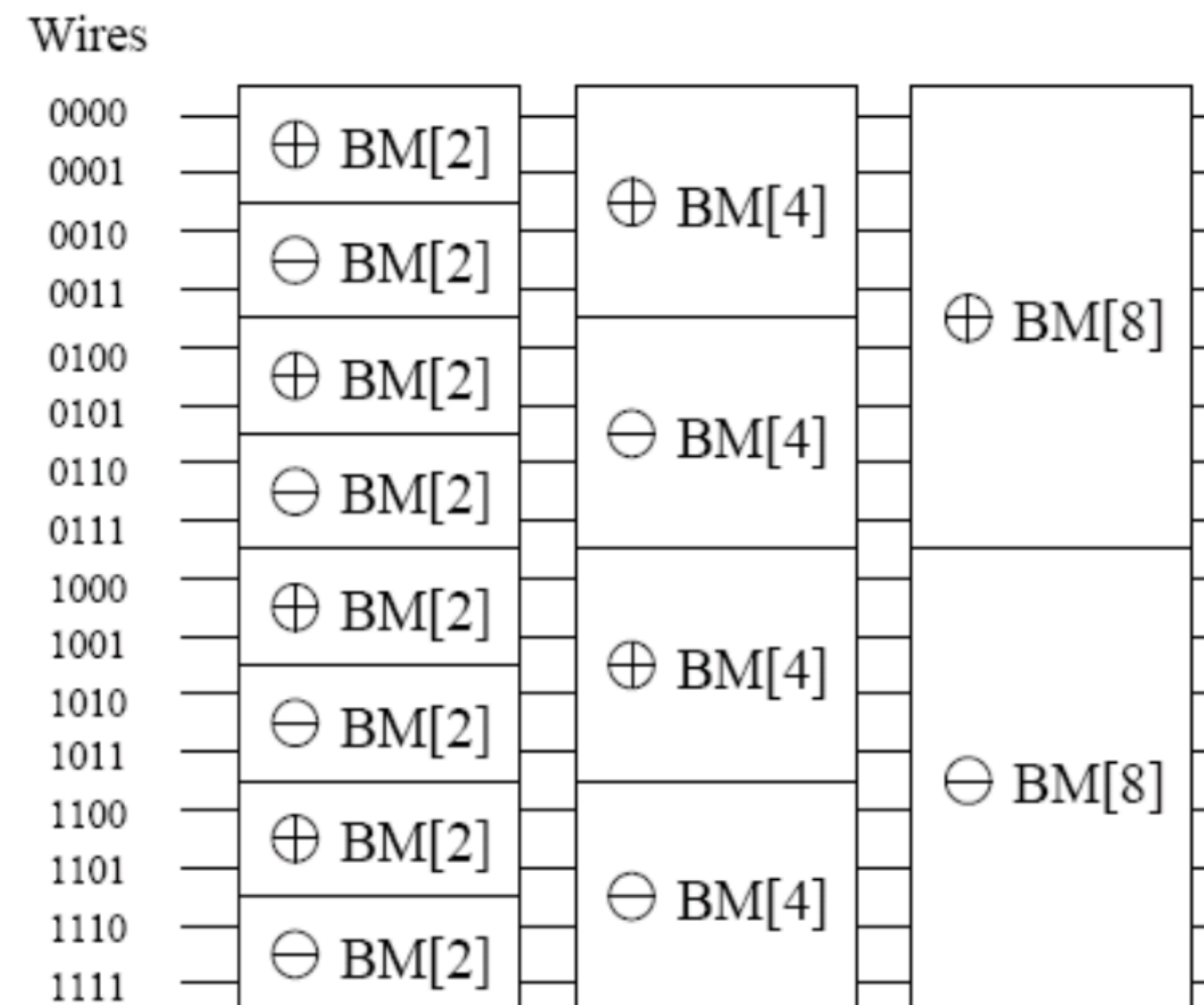
How do we sort an unsorted sequence using a bitonic merge?

Two steps

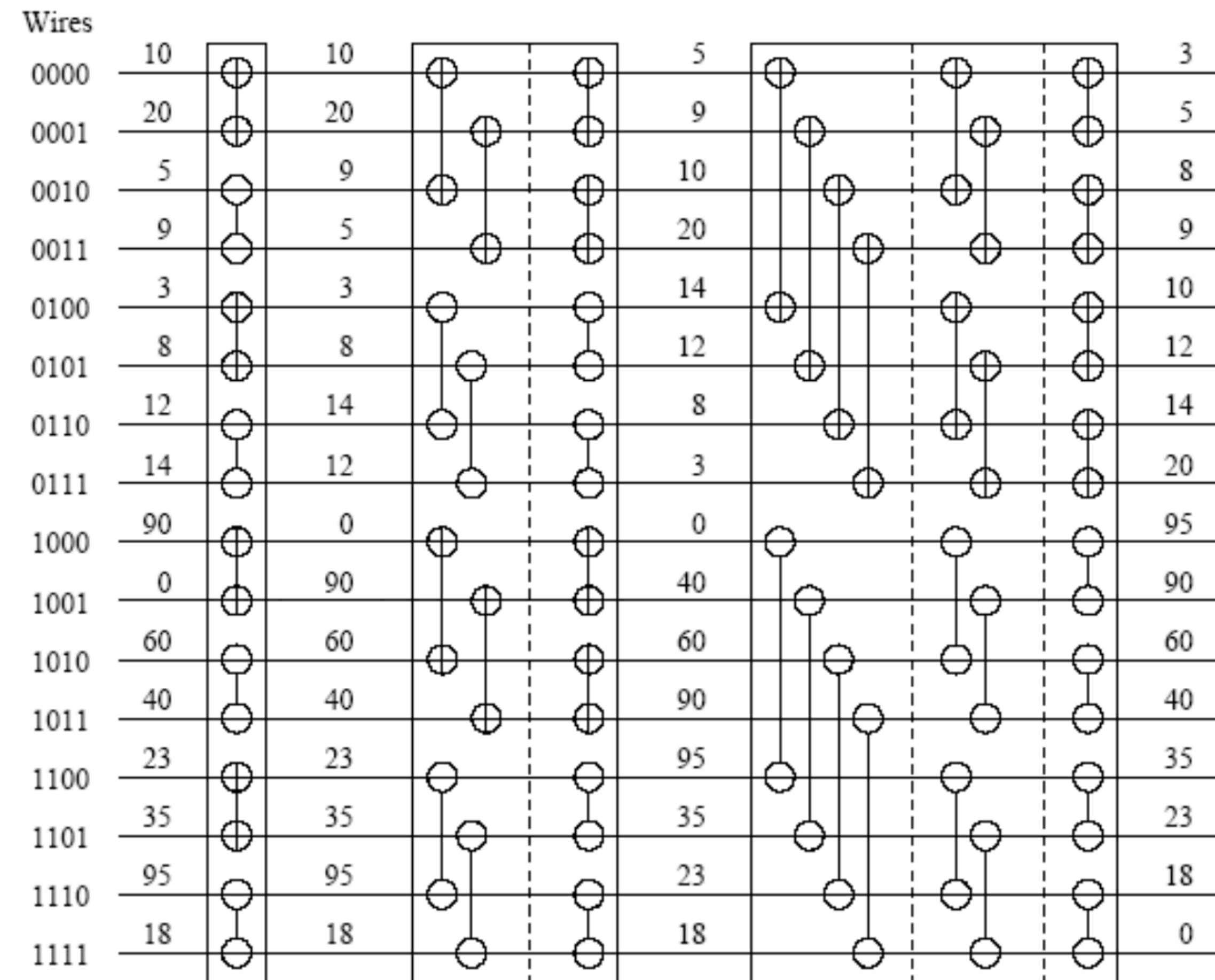
1. Build a bitonic sequence
2. Sort it using a bitonic merging network

Building a Bitonic Sequence

- **Build a single bitonic sequence from the given sequence**
 - any sequence of length 2 is a bitonic sequence.
 - build bitonic sequence of length 4
 - sort first two elements using $\oplus\text{BM}[2]$
 - sort next two using $\ominus\text{BM}[2]$
- **Repeatedly merge to generate larger bitonic sequences**
 - $\oplus\text{BM}[k]$ & $\ominus\text{BM}[k]$: bitonic merging networks of size k



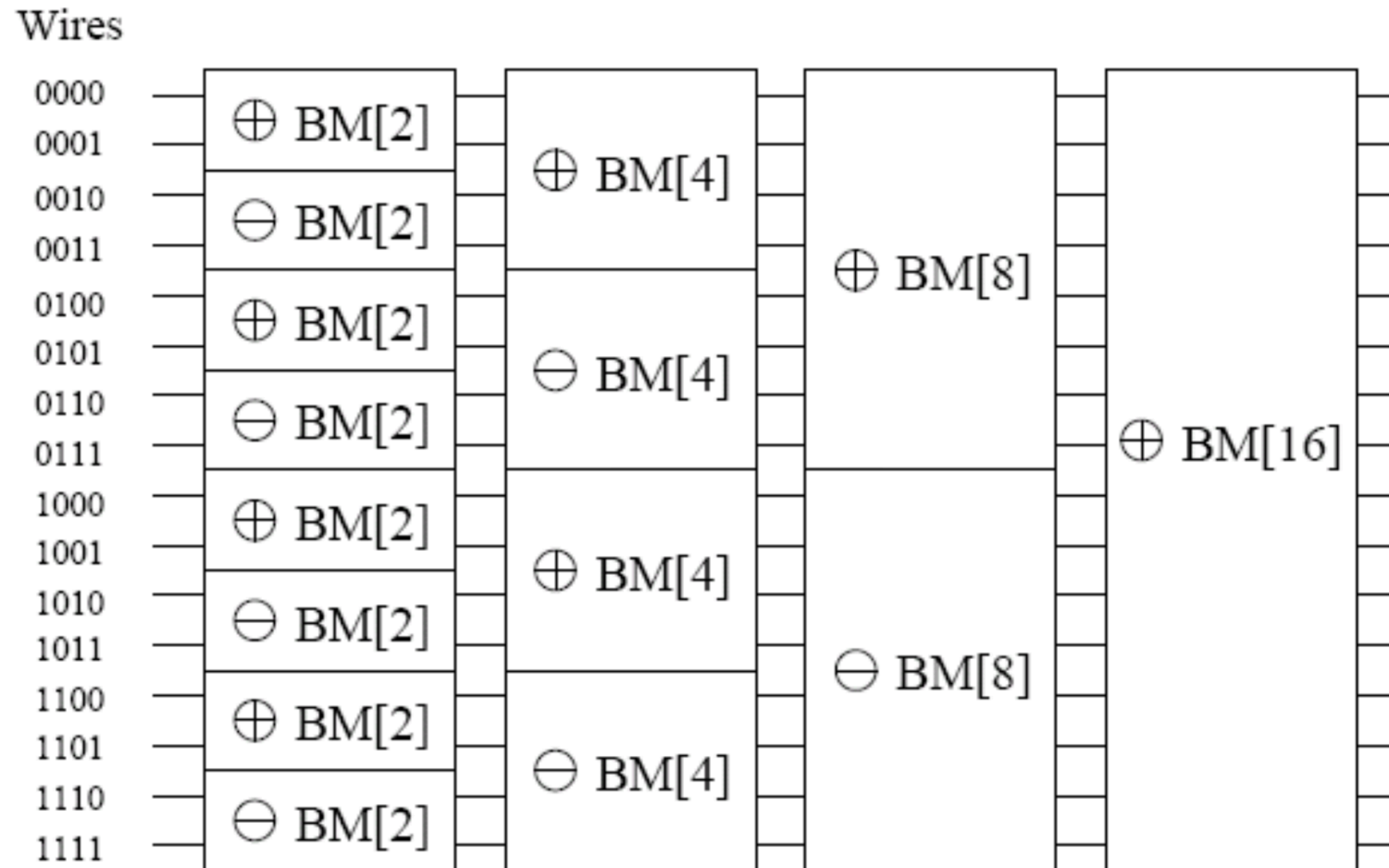
Building a Bitonic Sequence



Input: sequence of 16 unordered numbers

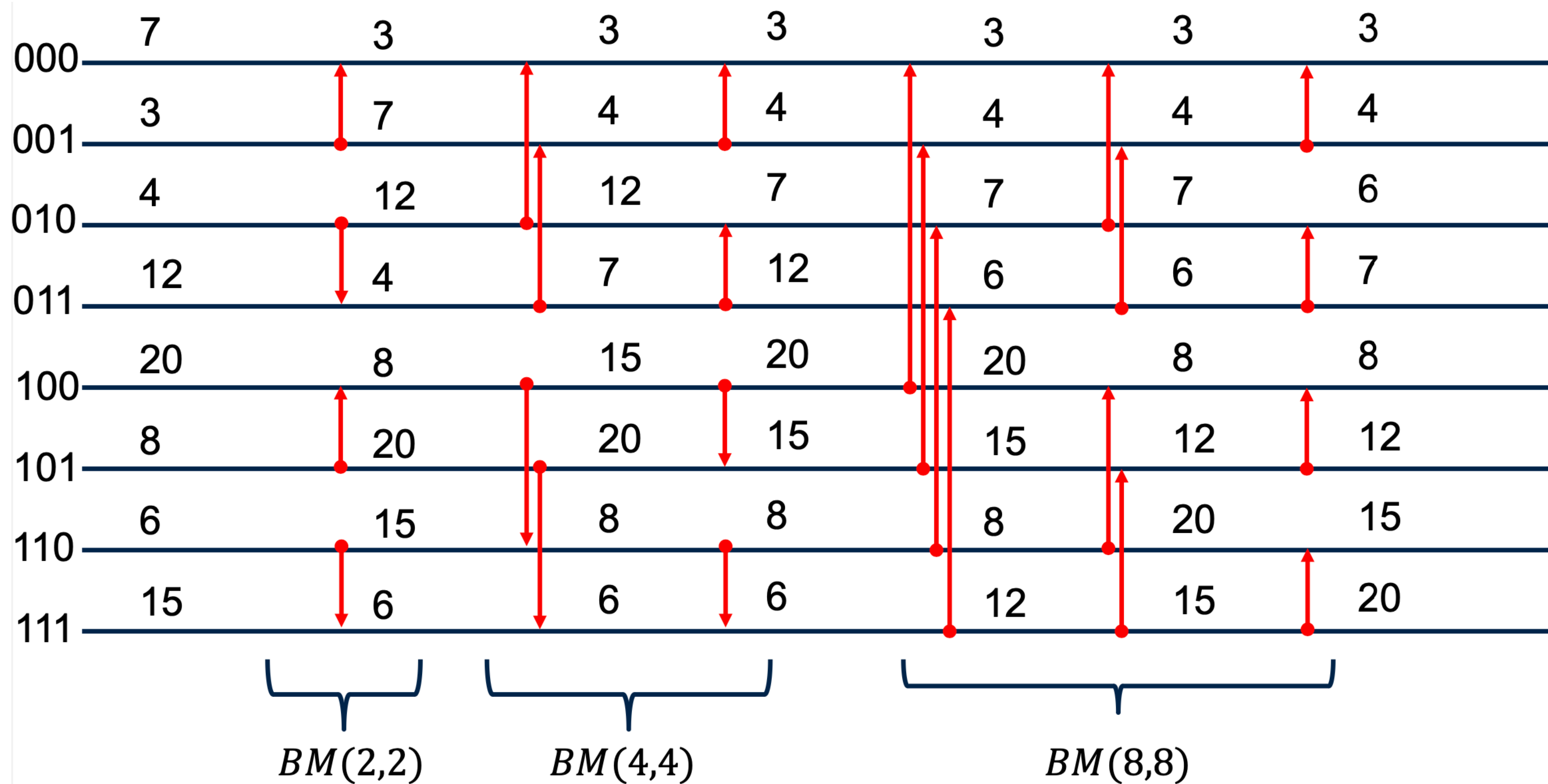
Output: a bitonic sequence of 16 numbers

Bitonic Sort, $n = 16$



- **First 3 stages create bitonic sequence input to stage 4**
- **Last stage ($\oplus$ BM[16]) yields sorted sequence**

Bitonic Sort Example



Complexity of Bitonic Sorting Networks

- **Depth of the network is $\Theta(\log^2 n)$**

$$\text{BM}(p, p) = O(\lg^2 p)$$

— $\log_2 n$ merge stages

— j^{th} merge stage depth is $\log_2 2^j = j$

—**depth** = $\sum_{j=1}^{\log_2 n} \log_2 2^j = \sum_{i=1}^{\log_2 n} j = (\log_2 n + 1)(\log_2 n) / 2 = \theta(\log^2 n)$

- **Each stage of the network contains $n/2$ comparators**
- **Complexity of serial implementation = $\Theta(n \log^2 n)$**

Bitonic Sort Pseudocode

```
for i=0 to (log p)-1 do
  for j=i downto 0 do
    if (i+1)st bit is 0 then
      Compare_Exchange↑ with (Rank XOR 2j)
    else
      CompareExchange↓ with (Rank XOR 2j)
  endfor
endfor
```


Generalizing Bitonic Sort

- $T(p, p) = O(\log^2 p)$
- Computation time = $O(\log^2 p)$
- Comm. Time = $O(\tau \log^2 p + \mu \log^2 p)$
- $n > p$?
- Step 1: Sort n/p local elements.
- Step 2: Run bitonic sort with CompareExchange replaced by MergeSplit
- Computation time = $O\left(\frac{n}{p} \log \frac{n}{p} + \frac{n}{p} \log^2 p\right)$
- Comm. Time = $O\left(\tau \log^2 p + \mu \frac{n}{p} \log^2 p\right)$